
Recherche MCTS multicœur par vagues

Conception, implémentation et évaluation dans LapZero-Chess

Rapport d'ingenierie

Mathieu Leclercq
19 septembre 2026

Version 1.0

Dernier commit cite : 0aaeeb1 (branche codex/mcts-multicore-waves)

Ce document raconte comment la recherche MCTS de LapZero-Chess est passée d'un cœur CPU à huit, pourquoi cette transformation a pris la forme de vagues, comment elle a été mise au point sous contrainte de correction, et ce que les mesures en disent. Il s'adresse à un lecteur familier du projet ou du domaine, et peut se lire d'un bout à l'autre comme un récit technique. Toutes les valeurs proviennent de campagnes conservées dans `out/multicore/`; les commits cités sont dans l'historique Git du dépôt.

Table des matières

Résumé	3
1 Introduction	4
1.1 Le projet LapZero-Chess	4
1.2 Le problème : le GPU attend le CPU	5
1.3 La question posée	5
1.4 Contributions	5
1.5 Ce que ce document ne couvre pas	6
1.6 Conventions de lecture	6
2 Le moteur juste avant le multicœur	6
2.1 Vue d'ensemble du moteur	6
2.2 Le MCTS batché	6
2.3 La table de transposition $h0$	7
2.4 Le batching par virtual loss già validé	7
2.5 Ce qui reste sériel	8
2.6 Contraintes imposées par le code	9
2.7 Le cahier des charges	9
2.8 La table du bot, rappel utile pour les mesures	9
3 Conception	10
3.1 Vue d'ensemble d'une vague	10
3.2 Pourquoi un arbre partagé	10
3.3 Le résultat de collecte	11
3.4 États de nœud et propriété	11
3.5 La garde RAI et le virtual loss	12
3.6 La table de transposition par snapshots	12
3.7 Le succès de table pendant la descente	13
3.8 Budget exact, permis et collisions	13
3.9 Exceptions, annulation et quiescence	13
3.10 Instrumentation et observabilité	14
3.11 Ce qui a été écarté, et pourquoi	14
4 Implémentation : chronologie et pièges	14
4.1 Tâche 1 : la référence et le chronométrage	14
4.2 Tâche 2 : l'évaluateur injectable et le banc self-play	15
4.3 Tâche 3 : le cache par snapshots	15
4.4 Tâche 4 : états atomiques et réservations RAI	15
4.5 Tâche 5 : le pool de workers persistant	15
4.6 Tâche 6 : le collecteur parallèle	15
4.7 Tâche 7 : le coordinateur et l'API	16
4.8 Tâche 8 : le stress et le self-play partagé	16
4.9 Tâche 9 : les bancs comparables	17
4.10 Tâche 10 : mesures et activation	17

5	Validation	18
5.1	Tester la concurrence sans outil d'analyse	18
5.2	Inventaire des tests	18
5.3	Les invariants	19
5.4	Les entrelacements forcés	19
5.5	Les tests de mutation	19
5.6	Ce que la validation ne couvre pas	19
6	Campagnes de mesure	20
6.1	Protocole et matériel	20
6.2	Aucune régression mono-worker	20
6.3	Débit et latence par nombre de workers	20
6.4	Répartition par phase et remplissage	21
6.5	Le régime réel de la boucle UCI	23
6.6	Non-régression du self-play	23
6.7	Qualité de recherche	24
6.8	La décision d'activation	25
7	Discussion	25
7.1	Pourquoi pas un facteur huit	25
7.2	Ce qui a marché, ce qui a déçu	26
7.3	Menaces sur la validité	26
7.4	Le coût d'ingénierie	27
7.5	Leçons	27
8	Conclusion et perspectives	28
8.1	Bilan	28
8.2	Ce que la production continue peut reprendre	28
8.3	Assainir l'appel à l'évaluateur	28
8.4	Autres pistes consignées	28
8.5	Mot de la fin	29
A	Chronologie des commits	29
B	Fichiers et responsabilités	30
C	Reproduction des campagnes	30
D	Glossaire	32

Résumé

Ce rapport retrace la conception, l'implémentation et l'évaluation d'une recherche MCTS multicœur dans LapZero-Chess, un moteur d'échecs de type AlphaZero entraîné sur une machine personnelle. Avant ce chantier, le moteur savait déjà regrouper plusieurs positions dans une seule inférence GPU grâce à un virtual loss de type Leela Chess Zero, mais un seul cœur CPU préparait les descentes. L'objectif était de faire préparer ces descentes par plusieurs cœurs sur un arbre partagé, sans corruption silencieuse et sans perte mesurable de qualité de recherche.

La solution retenue est une recherche *par vagues* : des collecteurs persistants descendent en parallèle avec des plateaux privés, déposent leurs feuilles sous réservation atomique, s'arrêtent à une barrière, puis le coordinateur exécute une seule inférence groupée, développe les feuilles et effectue les remontées de valeur. La correction repose sur des états de nœud atomiques, une garde RAI qui possède le chemin réservé, une table de transposition par snapshots et verrous rayés, et un budget exact de simulations. La production continue, qui ferait descendre les workers pendant les inférences, est restée hors périmètre : elle fera l'objet d'une décision séparée.

Le gain mesuré est modeste mais reproductible : de +7 % à +26 % de simulations par seconde selon la position, avec un plafond dès deux workers, un p95 de latence qui ne se dégrade pas, et une qualité non inférieure sur 2 500 puzzles. La raison de ce plafond est claire dans les mesures : l'inférence réseau occupe 96 à 99 % du temps, donc paralléliser la préparation CPU ne peut pas faire davantage. Le réglage `MCTS_WORKER_COUNT = 8` a été activé dans le bot UCI.

L'essentiel en dix points.

1. Avant le multicœur, chaque simulation payait une latence fixe d'environ 3,4 ms, et le débit était plat (286 à 299 simulations par seconde) quelles que soient la position et la profondeur. Le batching par virtual loss a fait monter ce débit à 320–850 selon la position et la taille de lot, et un lot de 8 a été validé pour la qualité.
2. Après le batching, l'inférence représente 96 à 99 % du temps de recherche. Le goulot restant est mécanique : un seul cœur prépare les descentes de chaque lot, et les lots partiels coûtent cher en latence.
3. L'architecture livrée sépare strictement deux phases : collecte parallèle sans écriture des statistiques d'arbre, puis traitement séquentiel après barrière (inférence unique, expansion, remontées).
4. La propriété d'une feuille passe par une transition atomique `Unexpanded` -> `Pending`. Une seule descente gagne ; les autres déclarent une collision, rendent leur permis et recommencent.
5. Le virtual loss devient visible pendant la sélection, pas après la collecte, et une garde RAI le restaure en cas de collision, d'exception ou de lot partiel. Aucun chemin d'erreur ne dépend d'une boucle manuelle.
6. La table de transposition ne rend plus jamais de pointeur vers une entrée modifiable : chaque lecture copie un *snapshot* sous verrou de bande, indexé sur l'index physique réel.
7. Le budget est exact : chaque simulation demandée produit exactement une remontée de valeur, et aucune réservation ne survit au repos.

8. La validation combine tests déterministes avec faux réseau, points de crochet pour forcer les entrelacements, tests de mutation, stress de 1 800 recherches et self-play partagé. Sous MSVC, faute de ThreadSanitizer, cette discipline est le filet principal.
9. Mesures : médiane de débit +8 à +26 % selon la position pour 8 workers, p95 de latence sous +5 % partout, et qualité non inférieure sur 2 500 puzzles (delta -0,28 point, IC95 [-0,8 ; +0,24]).
10. La production continue, l'assainissement de l'appel à l'évaluateur et le cas de l'arbre chaud restent des chantiers séparés, documentés en fin de rapport.

Mots-clés : MCTS, recherche parallèle, arbre partagé, virtual loss, table de transposition, C++17, ONNX Runtime, AlphaZero, test-driven development.

Comment lire ce document

Le rapport suit un ordre narratif. Le chapitre 1 pose le problème et la question. Le chapitre 2 décrit l'état du moteur juste avant le chantier, avec les mesures qui ont motivé la conception. Le chapitre 3 est le cœur conceptuel : chaque mécanisme y est justifié, alternatives écartées comprises. Le chapitre 4 raconte l'implémentation en dix tâches, avec les pièges rencontrés et les extraits de code décisifs. Le chapitre 5 décrit la stratégie de validation. Le chapitre 6 présente toutes les campagnes de mesure. Le chapitre 7 discute les résultats, leurs limites et les leçons. Le chapitre 8 conclut et ouvre les perspectives. Les annexes contiennent la chronologie des commits, l'inventaire des fichiers, les commandes de reproduction et un glossaire.

Les encadrés bleus résument chaque section importante, les encadrés ambre signalent des pièges vécus, et les encadrés gris décrivent les protocoles de mesure. Les valeurs numériques proviennent des fichiers `out/multicore/` conservés hors du dépôt (ils sont volumineux) ; les rapports agrégés cités, eux, sont commités.

1 Introduction

1.1 Le projet LapZero-Chess

LapZero-Chess est un moteur d'échecs de type AlphaZero, développé pour tenir sur une seule machine personnelle. L'architecture suit le schéma classique du papier de DeepMind [1] : un réseau résiduel prédit une politique de coups et une valeur de position, et une recherche arborescente de Monte-Carlo (MCTS) combine ces prédictions avec des statistiques de visite pour produire à la fois les coups joués et les cibles d'entraînement.

Le pipeline comporte trois étages. D'abord un pré-entraînement supervisé sur des parties de maîtres et de joueurs Lichess, qui donne au réseau les rudiments tactiques et positionnels. Ensuite une boucle de self-play : le moteur joue contre lui-même, chaque partie produit des couples (état, politique de visites, résultat) qui servent à réentraîner le réseau. Enfin un bot UCI, déployé sur Lichess, qui utilise le même moteur de recherche en partie réelle.

Le cœur du moteur est écrit en C++17 et exposé à Python par pybind11. Le réseau est exécuté par ONNX Runtime, sur GPU CUDA quand il est disponible, avec un repli CPU. Le MCTS, la génération de coups, la table de transposition et le code de self-play vivent tous dans ce cœur C++, ce qui permet d'atteindre environ 1,8 million de nœuds par seconde en perft mono-thread, et de garder la boucle d'entraînement en Python léger.

1.2 Le problème : le GPU attend le CPU

La recherche MCTS évalue le réseau sur des positions feuilles. Jusqu'en septembre 2026, chaque simulation déclenchait une inférence sur une seule position. La conséquence a été mesurée avec précision : le débit était de 286 à 299 simulations par seconde, indépendamment de la position, du nombre de coups légaux (14 en finale, 48 en milieu de jeu) et de la profondeur de l'arbre. Une telle insensibilité à la forme du travail dénonce un coût dominé par une latence fixe : environ 3,4 ms pour lancer un noyau GPU sur une position.

Le premier chantier, le batching par virtual loss, a groupé jusqu'à huit feuilles dans une seule inférence. Le débit est monté jusqu'à environ 850 simulations par seconde en ouverture, et un lot de 8 a été validé comme le meilleur compromis qualité-débit. Mais une fois ce travail fait, le profil du temps a changé de nature : l'inférence elle-même représente désormais 96 à 99 % du temps total, et le reste est dominé par une étape que le batching n'avait pas parallélisée, la préparation sérielle des descentes.

Concrètement, la boucle batchée est la suivante : pendant qu'un unique cœur descend l'arbre huit fois de suite pour collecter huit feuilles, le GPU ne fait rien ; puis, pendant que le GPU évalue le lot, ce cœur attend. Chaque vague paie donc deux attentes alternées. Un indice supplémentaire est le remplissage : les lots contiennent souvent 5 à 7 positions utiles au lieu de 8, parce qu'une collision ou une fin d'arbre ferme le lot en cours de route. Chaque appel GPU partiellement rempli coûte sa latence fixe pour moins de travail.

1.3 La question posée

Ce rapport répond à une question d'ingénierie précise : peut-on faire préparer les descentes par plusieurs cœurs CPU sur un arbre partagé, sans corruption silencieuse, sans casser le self-play, et pour quel gain réel de débit et de qualité ?

La difficulté n'est pas d'écrire une boucle parallèle, mais de rendre correcte la cohabitation de plusieurs descentes dans une même structure de données qui n'a jamais été conçue pour cela. L'arbre est muté en permanence (visites, valeurs, enfants), la table de transposition est un vecteur partagé, et le plateau d'échecs lui-même se modifie pendant la génération des coups. Chaque accès concurrent doit être soit rendu invisible, soit protégé par un protocole explicite. C'est ce travail d'architecture, plus que le gain brut, qui constitue le sujet de fond de ce document.

1.4 Contributions

Le chantier a produit quatre choses que ce rapport documente en détail.

1. **Une architecture par vagues** : les workers collectent en parallèle sans jamais écrire les statistiques de l'arbre, puis le coordinateur traite le lot entier séquentiellement. La correction est obtenue par séparation stricte des phases plutôt que par verrouillage généralisé.
2. **Cinq composants réutilisables et testables isolément** : un évaluateur injectable, une table de transposition par snapshots, une garde de réservation RAII sur des états de nœud atomiques, un exécuteur de workers persistant, et un collecteur de feuilles borné.
3. **Une instrumentation et un harnais de mesure** capables de séparer les temps par phase, de comparer des campagnes appariées, et de détecter une régression de qualité à 0, 1 point près sur 2 500 puzzles.
4. **Un verdict mesuré** : le multicœur apporte +8 à +26 % selon la position, plafonne dès deux workers, ne dégrade pas la queue de latence ni la qualité, et a été activé dans le bot UCI à huit workers.

1.5 Ce que ce document ne couvre pas

Trois sujets voisins sont volontairement laissés de côté. La *production continue*, qui ferait descendre les workers pendant les inférences et les remontées, est l'étape 2 de la conception ; elle n'est pas implémentée et les mesures de ce rapport servent précisément à décider si elle vaut son coût. L'*assainissement de l'appel à l'évaluateur* (forme de lot fixe, buffers réutilisés, quantification) est un chantier indépendant, documenté dans 2026-09-19-cout-calcul-cpu-gpu.md. Enfin la parallélisation du self-play n'est pas concernée : il parallélise déjà des parties indépendantes et n'a pas besoin d'un arbre partagé.

1.6 Conventions de lecture

Les mesures de débit sont exprimées en *simulations par seconde* (sims/s). Une simulation est une remontée de valeur complète, qu'elle ait coûté une inférence ou non (une simulation qui s'arrête sur un nœud terminal ou sur un succès de table n'appelle pas le réseau). Les trois positions de référence du banc sont appelées *ouverture*, *milieu* et *finale* ; leurs FEN sont données en annexe C. Le *remplissage* est le nombre de positions réellement évaluées par appel GPU. La *table de transposition* est notée TT, et la politique h_0 désigne la clé sémantique du cache décrite au chapitre 2. Les sessions de mesure varient de 15 à 30 % entre elles (horloge et thermique du GPU) : sauf mention contraire, seules les comparaisons intra-session sont utilisées, et les protocoles sont conçus pour annuler la dérive.

2 Le moteur juste avant le multicœur

En resume

Au moment d'ouvrir le chantier multicœur, le moteur batchait déjà les évaluations par paquets de huit grâce à un virtual loss de type Leela Chess Zero, et la table de transposition utilisait une clé sémantique h_0 . Le débit était passé de 286–299 à 320–850 simulations par seconde selon la position, mais l'inférence occupait désormais 96 à 99 % du temps. Tout le travail restant était donc d'un seul côté de la barrière : un unique cœur préparait les lots, sérieusement, et les lots partiels coûtaient leur latence fixe pour moins de positions utiles. Ce chapitre décrit cet état des lieux et les contraintes qu'il impose.

2.1 Vue d'ensemble du moteur

Le moteur se compose de quatre ensembles, représentés sur la figure 1. Le cœur C++ contient le plateau (Chessboard), le générateur de coups, le MCTS et sa table de transposition, ainsi que le SelfPlayManager qui fait jouer plusieurs parties en parallèle. Le réseau est un ResNet de 10 blocs et 128 filtres, avec des blocs d'attention par canal (Squeeze-and-Excitation), exporté en ONNX et exécuté par ONNX Runtime. Python orchestre l'entraînement, les tournois et le bot UCI. Les deux mondes communiquent par pybind11.

2.2 Le MCTS batché

Le MCTS est un PUCT classique. Pour un enfant i de parent p , le score de sélection est

$$\text{score}(i) = \underbrace{\frac{W_i}{N_i}}_{\text{exploitation}} + c_{\text{puct}} \cdot P_i \cdot \underbrace{\frac{\sqrt{N_p}}{1 + N_i + V_i}}_{\text{exploration}} \quad (1)$$

où N_i est le nombre de visites terminées, W_i la somme des valeurs, P_i le prior du réseau, V_i le nombre de descentes en cours (virtual loss) et $c_{\text{puct}} = 1,4$. Deux variantes locales

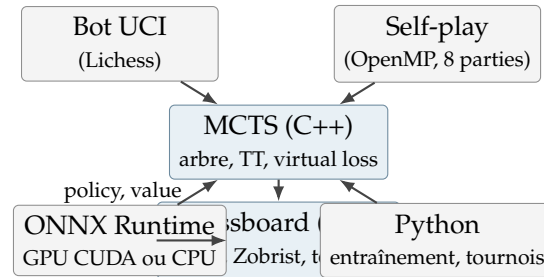


FIGURE 1 – Architecture du moteur. Le MCTS est partagé par le bot UCI et le self-play ; le chantier multicœur ne modifie que le premier, mais les abstractions communes (cache, réservations, évaluateur) servent aux deux.

comptent. D’abord le *First Play Urgency* : un enfant jamais visité hérite de la valeur du parent, diminuée d’une réduction proportionnelle à la somme des priors visités, ce qui évite les erreurs catastrophiques en début d’exploration. Ensuite l’expansion paresseuse : un nœud ne développe ses enfants qu’au moment où il devient feuille, et un succès de table permet d’éviter complètement l’appel réseau.

Le virtual loss mérite une précision, car c’est le mécanisme qui rend le batching possible. En sélectionnant le même score que ci-dessus avec $V_i > 0$ pour les nœuds déjà empruntés par une descente en cours, on décourage naturellement les collecteurs à converger vers la même feuille. Le compteur V est strictement séparé de N et de W : la valeur $Q = W/N$ reste la valeur honnête de la position, et seul le terme d’exploration est affecté. C’est le choix de Leela Chess Zero [5] ; l’alternative historique de Chaslot [7] ajoute une défaite fictive à la valeur, ce qui aurait ici contaminé Q et, indirectement, le First Play Urgency des frères non visités.

2.3 La table de transposition h_0

La table associe à une position une politique légale et une valeur, calculées par le réseau, afin d’éviter des inférences redondantes. Sa clé a fait l’objet d’un chantier indépendant. Le tenseur d’entrée du réseau contient huit plans d’historique, les plans de répétition, le compteur des cinquante coups et des drapeaux de contexte ; le hash Zobrist seul ne représente rien de tout cela, et un succès de table sur le seul Zobrist pouvait renvoyer une politique calculée dans un autre contexte.

La politique retenue, dite h_0 , protège exactement les grandeurs qui changent la valeur ou les coups légaux : le compteur de la règle des cinquante coups, la catégorie de répétition courante et le contexte courant (droits de roque, prise en passant, trait). Un probe vérifie ces champs dans un ordre fixe et classe chaque refus (RULE50_REJECT, CONTEXT_REJECT, HISTORY_REJECT) pour que les campagnes puissent les compter séparément. C’est cette politique qui sert de référence à toutes les mesures du présent rapport.

2.4 Le batching par virtual loss già validé

Le batching a été introduit en septembre 2026. Sa conception [2] est guidée par trois observations.

1. La lenteur n’était pas dans la génération de coups : le débit était identique entre une finale à 14 coups légaux et un milieu à 48.
2. Le remède correct était le virtual loss séparé de LC0, pas la défaite fictive de Chaslot, pour ne pas contaminer Q ni le FPU.
3. La collecte devait capturer le plateau à la feuille (tenseur, coups légaux, hash) de sorte qu’aucun rejou de chemin ne soit nécessaire ensuite, ni pour l’expansion ni pour l’annulation du virtual loss.

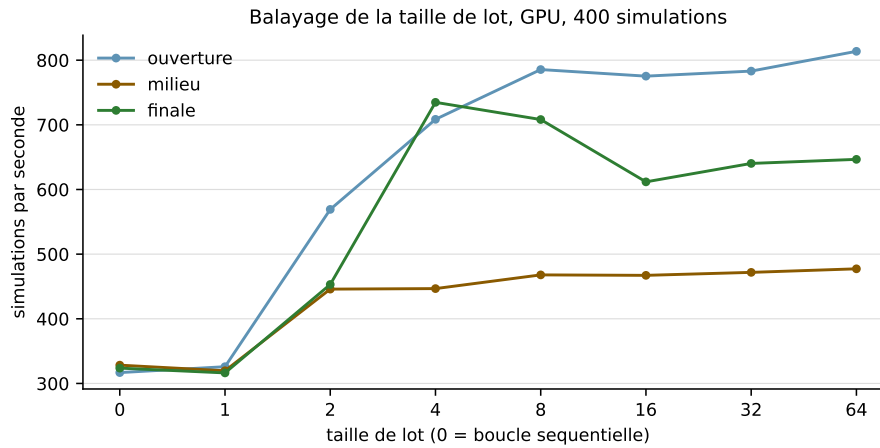


FIGURE 2 – Balayage de la taille de lot sur GPU, 400 simulations, une seule session (données : 2026-09-14-search-bench-batching-gpu.md). Le débit sature autour de 8 à 16 selon la position, et le remplissage plafonne vers 7 à 10 : l’arbre d’un seul arbre ne fournit pas plus de feuilles utiles.

Un lot se collecte en descendant l’arbre en posant V sur chaque nœud du chemin, en capturant les données à la feuille, puis en annulant les coups joués pour revenir à la racine. Une seule inférence évalue le lot, puis chaque feuille est développée et remontée depuis la capture. Le balayage de taille de lot sur GPU, réalisé le 14 septembre 2026, donne la courbe de la figure 2 et fixe le tableau suivant pour la position d’ouverture (700 simulations plus tard porteront des valeurs plus élevées, les mécanismes étant identiques).

Position	lot 0	lot 2	lot 4	lot 8	lot 16	lot 64
ouverture	316.8	569.2	708.5	785.5	775.3	813.7
milieu	328.3	445.9	446.7	467.9	467.2	477.3
finale	323.7	453.1	734.8	708.2	611.8	646.6

TABLEAU 1 – Débit en simulations par seconde selon la taille de lot (400 simulations, GPU, 5 passages). À lot 8, le remplissage réel vaut 6,4 en ouverture, 5,0 en milieu et 5,7 en finale.

Le lot 8 a été retenu : il offre presque tout le gain disponible, et sa qualité a été validée sur 2 500 puzzles contre le chemin séquentiel (batch 0 : 1 917 réussites ; batch 8 : 1 920 ; McNemar $p = 0,73$, quinze paires perdues contre dix-huit gagnées). Le batching est donc actif dans le bot UCI, le jeu local et le tournoi, tandis que le self-play, qui batchait déjà 512 positions entre parties, n’a pas changé.

2.5 Ce qui reste sériel

Le batching a résolu le problème de la latence d’inférence, mais il a laissé intacte la structure de la boucle de collecte. Cette boucle est sérielle pour trois raisons précises.

1. **Un seul plateau.** La descente utilise un unique Chessboard, celui de l’appel. Deux descentes simultanées sur le même plateau seraient impossibles : la génération des coups déplace temporairement des pièces.
2. **Des lots partiels.** Quand une descente retombe sur une feuille déjà réservée, la collecte s’arrête et le lot part avec moins de 8 positions. À 400 simulations, le remplissage moyen est de 4,9 à 6,9 selon la position.
3. **Des statistiques d’arbre non protégées.** Visites, valeurs et listes d’enfants sont lues et modifiées sans synchronisation adaptée à plusieurs écrivains concurrents.

La conséquence mesurée est un temps d'inactivité du GPU pendant la collecte, et un temps d'inactivité du CPU pendant l'inférence. Le graphique des phases (chapitre 6) montrera que l'inférence occupe 96 à 99 % du temps, et que la collecte plus les lots partiels représentent l'essentiel du reste.

2.6 Contraintes imposées par le code

La conception multicœur devait composer avec six contraintes non négociables, qui expliquent la plupart des choix du chapitre suivant.

1. **Un plateau complet par worker.** Chessboard n'est pas partageable, même pour une lecture : `isMoveSafe` déplace et restaure des pièces pour tester la légalité d'un coup. Chaque worker reçoit donc une copie complète du plateau (position, historiques, hash, tenseurs).
2. **Un arbre unique.** Plusieurs arbres par cœur dupliqueraient les évaluations et dilueraient la profondeur. Le bénéfice de la table et des statistiques mutualisées est le cœur de la force du moteur, et `update_root` réutilise le sous-arbre du coup joué en partie réelle.
3. **Une table partagée.** La table doit être lue et écrite par tous les collecteurs, et par le coordinateur pendant l'expansion. Sa structure actuelle (un vecteur avec des pointeurs bruts) ne garantit rien.
4. **Un self-play à préserver.** Le self-play utilise les mêmes fonctions MCTS depuis une région OpenMP à huit fils, mais sur des parties indépendantes. Ses performances et sa correction ne doivent pas régresser.
5. **C++17 et MSVC.** Pas de `std::jthread` ni de `std::barrier` : le rendez-vous des workers doit être construit avec mutex et variables de condition. Pas de ThreadSanitizer sous MSVC : la correction doit reposer sur la conception et les tests, pas sur un outil d'analyse dynamique.
6. **Un périmètre figé.** Les 119 plans du tenseur, le codage des coups, les règles du plateau, la politique `h0`, le cœur de recherche actuel à un worker et le self-play restent inchangés. Aucune nouvelle dépendance de test ni migration de langage.

2.7 Le cahier des charges

La spécification de conception [3] et le plan d'implémentation [4] fixent le cadre en dix tâches. L'architecture cible est une recherche *par vagues*, livrée en deux étapes compatibles : d'abord les vagues, où la collecte est parallèle mais le traitement est séquentiel après une barrière ; ensuite, seulement si les mesures le justifient, une *production continue* où les workers descendent pendant les inférences. La production continue n'est pas implémentée ici.

Les critères d'acceptation sont explicites et seront repris au chapitre 6 : une amélioration reproductible de la médiane de débit, un p95 de latence qui n'augmente pas de plus de 5 %, une qualité de recherche non inférieure à 1 point de pourcentage près (bootstrap apparié, marge unilatérale de -1 point), tous les invariants vérifiés et le self-play sans régression. Le réglage UCI par défaut ne bascule vers le multicœur que si tous ces critères sont franchis. Le plan prescrit un cycle uniforme pour chaque tâche : écrire le test en échec, implémenter le minimum, vérifier de façon ciblée, puis commiter les seuls fichiers concernés. Le chapitre 4 en raconte le déroulement.

2.8 La table du bot, rappel utile pour les mesures

Le bot UCI construit sa table avec `tt_size = 4_000_000` entrées, soit environ 4,16 Gio de RAM avec la structure actuelle (1 040 octets par entrée). Les campagnes de ce rapport utilisent une table de 8 192 entrées, comme les bancs précédents : à 700 simulations, elle suffit, et elle élimine la variable mémoire des comparaisons. Cette différence est consignée comme réserve

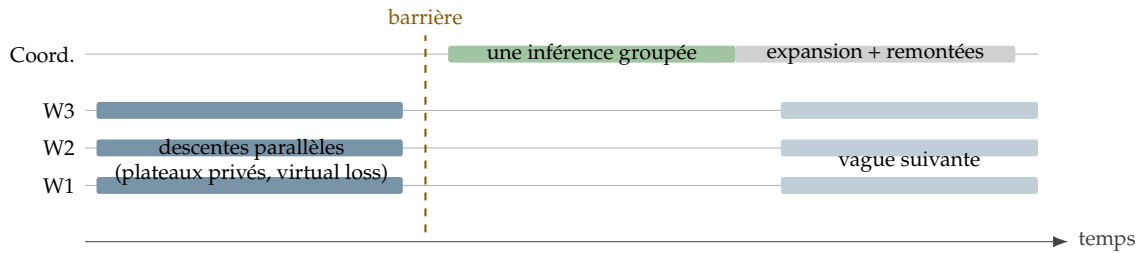


FIGURE 3 – Déroulement d’une vague. La collecte est parallèle et sans écriture des statistiques ; la barrière garantit que l’inférence, les expansions et les remontées voient un arbre stable.

au chapitre 7 : elle change le taux de succès de table en partie réelle, donc l’ampleur du gain transmissible.

3 Conception

En resume

La recherche par vagues sépare strictement deux temps. Pendant la collecte, plusieurs workers descendent en parallèle avec leurs propres plateaux et n’écrivent jamais les statistiques de l’arbre ; ils réservent les feuilles par une transition atomique et déposent des résultats immuables. Après la barrière, le coordinateur exécute une seule inférence, développe les feuilles et remonte les valeurs, séquentiellement. La correction tient à quatre mécanismes : des états de nœud atomiques, une garde RAI qui possède le chemin réservé, une table de transposition par snapshots sous verrous rayés, et un budget de simulations exact. La production continue, les verrous par nœud et la déduplication des évaluations sont explicitement écartés à ce stade.

3.1 Vue d’ensemble d’une vague

Une vague est la plus petite unité de travail du système. Le coordinateur choisit un nombre de permis égal à $\min(\text{lot}, \text{simulations restantes})$, puis les workers actifs descendent l’arbre en parallèle. Un worker peut enchaîner plusieurs tentatives par vague, de sorte que deux ou quatre workers peuvent remplir un lot de huit. Quand tous les collecteurs sont arrêtés, la barrière s’ouvre : le coordinateur fusionne les résultats dans un ordre stable, traite les terminaux, lance au plus une inférence groupée, développe les feuilles et effectue les remontées. La vague suivante ne commence qu’après publication complète de ces modifications.

3.2 Pourquoi un arbre partagé

L’alternative naturelle, un arbre par cœur, a été écartée pour trois raisons. D’abord elle duplique les évaluations : deux arbres indépendants redécouvrent la même position et paient deux inférences. Ensuite elle dilue la profondeur : huit arbres à 700 simulations explorent chacun un arbre de 700 nœuds au lieu d’un arbre de 5 600. Enfin elle complique fortement la réutilisation de racine : après un coup joué, le sous-arbre du coup doit être retrouvé dans chaque arbre, ou perdu.

Le self-play, lui, fonctionne déjà par parties indépendantes : ses huit fils OpenMP ne partagent pas d’arbre, et c’est très efficace parce que les parties divergent naturellement. La recherche UCI n’a pas cette chance : une seule position à analyser, donc un seul arbre. C’est précisément ce cas que les vagues traitent.

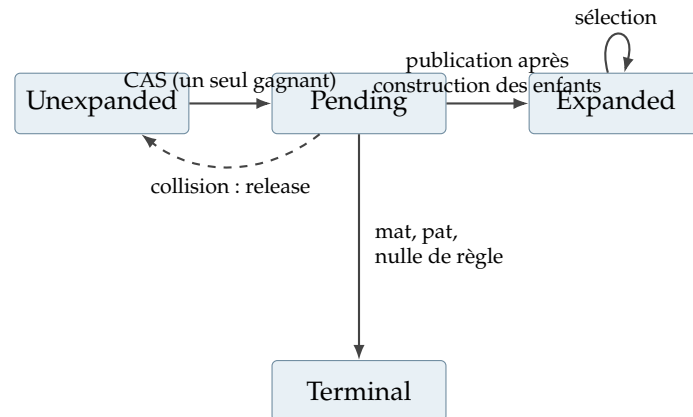


FIGURE 4 – Machine à états d'un nœud. La transition vers Expanded est une publication : les enfants sont entièrement construits avant que l'état ne devienne visible. La transition en pointillés est le retour arrière d'une collision ou d'une exception.

3.3 Le résultat de collecte

Une descente produit un objet déplaçable, `LeafWork`, qui ne contient aucun pointeur vers le plateau local. Ses champs :

- le type de résultat : `Network`, `Terminal` ou `Collision`;
- le nœud réservé et la garde `PathReservation` complète qui porte le virtual loss de tout le chemin;
- la clé de cache et les coups légaux, capturés à la feuille;
- le tenseur du réseau, uniquement pour une vraie feuille;
- la valeur terminale, lorsqu'elle est déjà connue.

Le résultat transfère au coordinateur la responsabilité de terminer ou d'annuler la réservation exactement une fois. Cette propriété est ce qui rend les chemins d'erreur simples : la destruction de la garde suffit à tout nettoyer.

3.4 États de nœud et propriété

Chaque nœud porte un état atomique à quatre valeurs :

État	Signification
Unexpanded	nœud jamais réservé, ses enfants n'existent pas
Pending	une descente le possède ; ses enfants se construisent
Expanded	liste d'enfants complète et publiée
Terminal	mat, pat, nulle de règle ou matériel insuffisant

La propriété s'acquiert par une comparaison-échange `Unexpanded -> Pending`. Un seul worker gagne ; les autres déclarent une collision. À aucun moment deux descentes ne peuvent développer le même nœud, ce qui résout le piège historique du batching : deux jeux d'enfants construits en parallèle sur la même feuille. L'état `Expanded` n'est publié qu'après un swap de la liste d'enfants complète, avec une écriture `release` ; un lecteur fait d'abord un chargement acquies de l'état, puis lit la liste. Aucun lecteur ne peut donc observer une liste partiellement construite.

3.5 La garde RAI et le virtual loss

Le virtual loss doit être visible pendant la sélection, pas après l'arrivée à la feuille. Chaque worker ajoute donc la racine puis chaque enfant choisi à sa garde PathReservation avant de continuer la descente. Deux workers peuvent encore choisir le même enfant simultanément; le virtual loss réduit cette probabilité mais ne la supprime pas. La garantie de correction reste la transition exclusive vers Pending.

La garde possède deux choses : le chemin réservé (un vecteur de pointeurs) et, le cas échéant, le nœud en Pending dont elle est propriétaire. Sa destruction libère tout exactement une fois.

```

1  bool PathReservation::try_claim(MCTSNode* node) {
2      if (node == nullptr || m_owned_pending != nullptr) return false;
3      NodeState expected = NodeState::Unexpanded;
4      if (!node->state.compare_exchange_strong(
5          expected, NodeState::Pending,
6          std::memory_order_acq_rel, std::memory_order_acquire)) {
7          return false; // collision : un autre collecteur possède le nœud
8      }
9      m_owned_pending = node;
10     return true;
11 }
12
13 void PathReservation::publish(NodeState state) noexcept {
14     if (m_owned_pending == nullptr) return;
15     m_owned_pending->state.store(state, std::memory_order_release);
16     m_owned_pending = nullptr; // la garde ne peut plus annuler
17 }
18
19 void PathReservation::release() noexcept {
20     if (m_owned_pending != nullptr) {
21         NodeState expected = NodeState::Pending;
22         m_owned_pending->state.compare_exchange_strong(
23             expected, NodeState::Unexpanded,
24             std::memory_order_release, std::memory_order_relaxed);
25         m_owned_pending = nullptr;
26     }
27     for (MCTSNode* node : m_path) {
28         node->n_in_flight.fetch_sub(1, std::memory_order_relaxed);
29     }
30     m_path.clear();
31 }

```

Listing 1 – La réservation, coeur de la garde RAI. La réservation enregistre le pointeur avant d'incrémenter le compteur, de sorte qu'une allocation ratée ne perd aucun compteur. La publication désarme la propriété avant de pouvoir annuler un état publié.

Trois propriétés de cette garde méritent d'être soulignées. D'abord, `reserve()` enregistre le pointeur dans le vecteur avant d'incrémenter le compteur atomique : si l'allocation échoue, aucun compteur n'est perdu. Ensuite, `release()` ne remet en `Unexpanded` que le nœud dont la garde est propriétaire et qui n'a pas été publié; si un autre worker a repris le nœud entre-temps, le CAS échoue et rien n'est écrasé. Enfin, la publication désarme la garde : un `release()` ultérieur ne peut plus annuler un état déjà visible.

3.6 La table de transposition par snapshots

La table ne rend plus jamais de pointeur vers une entrée. Un `probe()` copie la valeur et les seules entrées de politique utiles dans un snapshot local, sous le verrou d'une bande; un `store()` prépare ses données puis remplace l'entrée sous le même verrou. L'index est le hash modulo la taille, et la bande est dérivée de l'index physique réel, pas d'un second modulo indépendant sur le hash :

```

1  std::size_t EvaluationCache::index_for(const EvaluationCacheKey& key) const {
2      return key.position_hash % m_entries.size();

```

```

3 }
4 std::size_t EvaluationCache::stripe_for(std::size_t index) const {
5     return index % m_stripes.size();    // 4096 bandes au plus
6 }

```

Listing 2 – Index et bande. La bande est dérivée de l’index réellement accédé, ce qui garantit que deux clés qui s’écrasent dans la même entrée prennent toujours la même bande.

L’ordre des rejets est celui de la sémantique `h0` : à hash égal, on vérifie le compteur des cinquante coups, puis le contexte courant, puis l’historique. L’attention à cet ordre n’est pas cosmétique : il détermine la classification des refus dans les compteurs, donc la lisibilité des campagnes. Un snapshot pris avant un écrasement reste valide ; c’est testé explicitement.

3.7 Le succès de table pendant la descente

Sur un succès de table, le worker qui possède le Pending copie le snapshot, construit les enfants localement, publie Expanded et *poursuit la même descente*. Ce comportement est celui du `select_leaf` historique, et il a été conservé volontairement : un succès de table ne consomme pas une simulation et ne provoque pas de remontée à lui seul. Les autres workers ne lisent la liste d’enfants qu’après avoir observé Expanded par une lecture acquire.

Les nœuds terminaux, eux, sont détectés avant toute consultation de la table. C’est une garantie de sémantique : une nulle par répétition ou par règle des cinquante coups dépend de l’historique complet, et ne doit jamais être servie par une entrée de cache calculée ailleurs.

3.8 Budget exact, permis et collisions

Le budget est la contrainte la plus facile à violer en silence. Trois règles le garantissent.

1. Chaque vague dispose de `slots = min(lot, simulations restantes)` permis, et un CAS décrémente un compteur disponible avant toute tentative. Un permis n’est consommé que par un résultat final (terminal ou feuille réseau) ; une collision le rend.
2. Un compteur global d’essais borne la vague à $4 \times \text{slots} + \text{workers tentatives}$, pour qu’un arbre très collisionné ne fasse pas boucler les workers indéfiniment.
3. Si les collisions empêchent de remplir le lot, le coordinateur exécute un lot partiel : le progrès est toujours garanti, et une vague vide est traitée comme une erreur de logique, pas ignorée.

À la fin d’un appel réussi, le nombre de remontées neuves est exactement égal au nombre de simulations demandé. Les compteurs distinguent les positions évaluées, les appels physiques, les succès de table et les collisions, ce qui permet de vérifier cette égalité sur chaque campagne.

3.9 Exceptions, annulation et quiescence

Toute exception, qu’elle vienne d’un worker, de l’évaluateur ou du coordinateur, suit le même chemin : publication de l’annulation, réveil de tous les workers, attente de leur retour, annihilation des gardes encore actives, restauration des Pending non publiés, puis relance de l’exception. Le destructeur de la garde fait le gros du travail ; le coordinateur ne fait que détruire les résultats qu’il détient encore.

Au repos, l’arbre doit être quiescent : aucun Pending, aucun `n_in_flight` non nul, chaque réservation terminée ou annulée exactement une fois. Ces invariants sont vérifiés par un parcours d’observabilité, `inspect_tree`, qui détaille aussi les enfants dupliqués, les parents incohérents, les priors qui ne somment pas à 1 et les statistiques non finies.

3.10 Instrumentation et observabilité

Les phases mesurées sont la sélection, la préparation du tenseur et de la clé, la lecture et l'écriture de table, l'attente de verrou, la copie du plateau, l'assemblage du lot, l'inférence, l'expansion, les remontées et l'attente des workers. Le chronométrage est optionnel, désactivé par défaut, et chaque thread écrit dans son propre accumulateur, fusionné après la recherche. Les définitions sont volontairement inclusives : le rapport ne prétend pas mesurer des noyaux GPU, il mesure des temps d'appel.

3.11 Ce qui a été écarté, et pourquoi

Piste écartée	Raison
Production continue	Deuxième étape, à décider après mesure. Elle exige une synchronisation des statistiques et de la structure d'arbre que la barrière rend inutile.
Un verrou par nœud	Consommation mémoire et contention pour un bénéfice inférieur à la transition CAS, qui n'utilise que 8 bits d'état.
Atomiques sur N et W	Le protocole de lecture cohérente de Q aurait été plus complexe que la barrière, sans gain mesurable.
Déduplication des évaluations	Deux nœuds logiques différents peuvent demander la même position ; dédupliquer est un chantier d'optimisation distinct, hors du plan.
Réglage de c_{puct}	Changer la recherche pendant qu'on change l'ordonnancement rendrait toute mesure ininterprétable.
Softmax sur les seuls coups légaux	Optimisation indépendante, notée au backlog.

4 Implémentation : chronologie et pièges

En resume

Le chantier a suivi dix tâches, chacune écrite en cycle test d'abord : un test rouge ciblé, une implémentation minimale, une vérification, un commit. Les fondations (chronométrage, évaluateur injectable, cache par snapshots, réservations RAIL, pool persistant) précèdent la collecte parallèle, le coordinateur, le stress, les bancs et enfin les mesures. Les deux incidents marquants sont un contrat de dimensions silencieux découvert par le banc self-play, et un test d'équivalence du pool chaud dont la première version comparait la mauvaise référence. Ce chapitre raconte l'ordre réel des découvertes, avec les commits correspondants.

4.1 Tâche 1 : la référence et le chronométrage

Première tâche, première discipline : mesurer avant de modifier. La référence a été capturée avec le binaire de départ, puis dix phases de recherche ont été instrumentées : sélection, préparation du tenseur et de la clé, lecture et écriture de table, attente de verrou, copie du plateau, assemblage, inférence, expansion, remontées, attente des workers. Le chronométrage est désactivé par défaut et sans coût mesurable dans ce mode : quand le pointeur de mesure est nul, aucune horloge n'est lue.

La campagne de référence a donné la répartition qui a orienté tout le reste : l'inférence représente environ 98 % du temps. C'est cette mesure, plus que toute intuition, qui a fixé l'objectif réaliste du chantier multicœur : paralléliser au mieux 2 % du temps, et réduire les lots partiels. Commit 4b4f92f.

4.2 Tâche 2 : l'évaluateur injectable et le banc self-play

Le MCTS ne connaissait que `ONNXEvaluator`. Une interface `Evaluator` a été extraite, avec une seule méthode `evaluate_batch`; `ONNXEvaluator` en hérite sans changer de comportement, et un `ControlledEvaluator` de test permet de simuler des échecs, des lots de tailles choisies et des blocages. En parallèle, les copies de `Chessboard` ont été verrouillées par des tests sur des histoires légales concrètes : roque, prise en passant, promotion, répétition, avec comparaison de la FEN, du Zobrist, du tenseur et des historiques après copie, coup joué et annulé.

Ce travail a aussi produit le banc `selfplay_phase_bench`, qui rejoue la structure du self-play partagé : huit racines privées, cent tours de collecte sous OpenMP, puis une évaluation et des remontées séquentielles. C'est ce banc qui a servi plus tard de détecteur de régression. Deux accrocs mineurs y ont été corrigés : une condition de validation inversée dans le nouvel outil en ligne de commande, et la découverte que l'exécutable autonome a besoin du répertoire `torch/lib` dans son `PATH` pour trouver `cublasLt`. Commit `fe63dbe`.

4.3 Tâche 3 : le cache par snapshots

La table de transposition est passée derrière une classe `EvaluationCache`. Le point de bascule est le type de retour : le probe ne rend plus un pointeur, mais une copie. Pour éliminer le verrou global, l'index est découpé en au plus 4096 bandes, dérivées de l'index physique. Les tests de collision forcent deux writers et deux readers sur des clés qui ciblent la même entrée, et vérifient qu'un snapshot ne mélange jamais deux politiques. La migration a été menée en deux temps : remplacer les types d'abord, compiler rouge, puis corriger chaque usage listé par le compilateur.

Le surcoût mesuré face au réseau est minuscule : environ 0,27 à 0,33 ms d'attente de verrou et 0,45 à 0,50 ms d'accès à la table pour une recherche de 0,67 à 1,46 seconde. Sur le faux réseau, où ce coût n'est plus masqué, la collecte passe de 0,79 à 1,14 ms pour 100 vagues, soit environ 3,5 microsecondes ajoutées par vague. Commit `a0a182d`.

4.4 Tâche 4 : états atomiques et réservations RAII

L'ancien booléen `is_terminal` a disparu au profit de l'énumération d'états décrite au chapitre 3, et le compteur `n_in_flight` est devenu un entier atomique. La classe `PathReservation` possède le chemin et le Pending éventuel. Un test de mutation a été pratiqué : en neutralisant volontairement la décrémentation du compteur, le test de fuite échoue comme prévu, puis le code correct est restauré. La garde a tenu cinquante exécutions concurrentes. Commit `a303d96`.

4.5 Tâche 5 : le pool de workers persistant

`SearchExecutor` crée les threads au premier appel et les rendort entre les vagues sur une variable de condition et un numéro de génération. C'est le rendez-vous C++17 demandé par les contraintes : pas de `std::jthread`, pas de `std::barrier`. Les tests vérifient que les identités de threads survivent aux appels (8, puis 2, puis 8), qu'une exception de worker est relayée après le retour de tous, et qu'un appel ultérieur reste possible après une exception. Le pool ne grandit qu'au repos. Commit `540e5ee`.

4.6 Tâche 6 : le collecteur parallèle

Le fichier `mcts_wave.cpp` contient la descente concurrente. Chaque worker a son `WorkerContext` (plateau complet, résultats locaux, chronomètre). La boucle de collecte réserve la racine, sélectionne avec le PUCT existant, réserve l'enfant immédiatement, applique le coup sur le plateau privé et recommence. Les points décision sont les suivants.

```
1 // ... descente : etat du noeud, terminal par les regles du plateau ...
2 if (is_rule_terminal(context.board)) {
```

```

3  if (state == NodeState::Pending) return collision();
4  if (state == NodeState::Unexpanded) {
5      if (!work.reservation.try_claim(node)) return collision();
6      work.reservation.publish(NodeState::Terminal);
7  }
8  work.kind = LeafKind::Terminal;
9  return work;
10 }
11 if (state == NodeState::Unexpanded) {
12     if (!work.reservation.try_claim(node)) return collision();
13     legal_moves = context.board.getLegalMoveIndices();
14     if (legal_moves.empty()) {
15         work.reservation.publish(NodeState::Terminal);
16         return work;    // mat ou pat, valeur terminale
17     }
18     key = make_cache_key(context.board);
19     probe = probe_tt(key, &context.timing);
20     if (probe.status == TTProbeStatus::HIT) {
21         children = make_children_from_probe(node, probe);
22         node->children.swap(children);    // publication complete
23         work.reservation.publish(NodeState::Expanded);
24         continue;    // meme descente
25     }
26     // vraie feuille : capturer tenseur, coups, cle, transferer la garde
27     context.board.getAlphaZeroTensor(work.tensor);
28     work.kind = LeafKind::Network;
29     return work;
30 }
31 // ... noeud Expanded : selection PUCT, reserve de l'enfant, coup joue ...

```

Listing 3 – Extrait de la boucle de collecte. Le terminal est testé avant toute lecture de table ; le CAS vers Pending est la seule garantie de propriété ; un succès de table publie les enfants et poursuit la descente, comme le faisait `select_leaf`.

Les permis bornent la vague : `slots = min(lot, restant)`, un CAS décrémente `available`, une collision le rend, un résultat final le consomme. Le nombre de tentatives est plafonné à $4 \times \text{slots} + \text{workers}$. Les vectors de résultats restent locaux à chaque worker et sont fusionnés après le rendez-vous dans un ordre stable (`worker`, `ordinal`). Deux tests décisifs ont été écrits à ce moment : la comparaison du tenseur capturé avec le tenseur obtenu en rejouant réellement l'historique, et la publication atomique depuis la table avec un point de crochet avant publication. Cent exécutions concurrentes ont validé l'ensemble. Commit 2772087.

4.7 Tâche 7 : le coordinateur et l'API

`run_search_waves` orchestre les vagues : copie du plateau racine dans les seuls contextes actifs, collecte, fusion, une inférence si au moins une feuille réseau, validation des dimensions et des valeurs finies, puis expansion et remontées dans l'ordre. Les compteurs nouveaux (`waves`, `leaf_collisions`, `completed_simulations`) sont exposés côté Python. Le paramètre `worker_count` est ajouté aux deux points d'entrée publics, avec une valeur par défaut de 1 qui préserve tous les appelants existants ; une configuration multicœur avec un lot nul est refusée avant toute mutation.

Le verrou de session est désormais tenu pendant tout l'appel, pas par simulation : les workers détiennent des pointeurs bruts vers l'arbre, donc `update_root` ne peut pas détruire la racine sous eux. Les tests intégrés couvrent les workers 2, 4 et 8, les lots 1 et 8, les budgets 0, 1, 3, 8, 17 et 100, les racines terminales, les erreurs de dimensions de l'évaluateur, l'immobilité des workers pendant une inférence bloquée, et la sérialisation de `update_root` par un évaluateur retenu. Commit 3b42dd2.

4.8 Tâche 8 : le stress et le self-play partagé

Cette tâche ne changeait pas l'algorithme : elle cherchait à casser le nouveau noyau. Trois positions, workers {2, 4, 8}, lots {1, 8}, cent recherches par configuration, en alternant racines

neuves et réutilisées, puis en alternant $8 \rightarrow 2 \rightarrow 1 \rightarrow 4$: 1 800 recherches et 115 200 simulations, invariants vérifiés après chaque appel. Le self-play partagé a été exercé avec huit racines privées sur une instance MCTS et une table minuscule, puis avec deux parties complètes du manager.

Le contrat de dimensions, silencieux

Le test `selfplay_shared_core` a terminé ses deux parties puis échoué : le faux évaluateur recevait un vecteur d'entrée plus grand que `batch_size × 119 × 64`. La cause était un buffer d'entrée réutilisé, redimensionné une fois pour le nombre maximal de parties, mais jamais ramené à la taille réelle du lot en cours de traitement. Un vrai évaluateur ONNX ne se plaint pas de cette différence : le contrat de dimensions n'était vérifié que par le faux réseau, et c'est précisément pour cela que le bug a été vu.

Deux corrections ont suivi : redimensionner le buffer juste avant chaque appel dans les deux chemins du manager, et faire dépendre `SelfPlayManager` de l'interface `Evaluator` plutôt que de la classe ONNX concrète, ce qui était la seule adaptation de production nécessaire. Deux tests de mutation ont été pratiqués : neutraliser la remise à zéro de `Pending` doit faire échouer le test d'exception, et neutraliser le CAS de propriété doit faire échouer le test de collision. Commits `ef327e2` et `9ad2dcb`.

4.9 Tâche 9 : les bancs comparables

Les bancs existants mesuraient un arbre neuf par mesure, sans notion de workers ni de pool. Ils ont gagné quatre capacités : un balayage de `-worker-counts`, un mode `-warmup-pool` qui réutilise l'objet MCTS entre les mesures en remettant arbre et table à froid hors chronométrage, une jambe GPU pour le banc de puzzles avec `-search-workers` et `-search-seconds`, et un sidecar de contexte par CSV (hash du modèle et du banc, budget, provider, workers, critère figé). La comparaison de campagnes a reçu son propre module, `multicore_comparison.py` : refus des doublons, des lignes manquantes et des métadonnées incompatibles, bootstrap apparié à 20 000 tirages, paires discordantes, McNemar, et verdict formel. Une passe de durcissement a ensuite ajouté le statut d'échec de campagne quand un puzzle est en erreur, les bornes exactes des verdicts, et l'alternance de l'ordre des workers entre passages. Commits `7bde9e3`, `7cef664` et `9840fa8`.

Le premier test d'équivalence du pool chaud était faux

Pour vérifier qu'un `clear_evaluation_cache()` ramène bien un objet au même état qu'un objet neuf, un test comparait la deuxième recherche de l'objet réutilisé à la deuxième recherche d'un objet neuf. Il a échoué proprement, et pour la bonne raison : la deuxième recherche de l'objet neuf bénéficiait de sa propre table déjà remplie, avec dix succès de table contre zéro après le clear. La référence correcte est la *première* recherche d'un objet neuf. Une fois corrigé, politiques et compteurs sont identiques au bit près, ce qui valide à la fois le clear et la conception du banc.

4.10 Tâche 10 : mesures et activation

La dernière tâche n'ajoute pas de mécanisme : elle exécute les campagnes, compare l'ancien et le nouveau binaire, mesure la qualité, rédige le rapport de résultats et active le réglage UCI. Le chapitre 6 détaille toutes les mesures. Commit `412b5d4` pour l'activation, `0aaeeb1` pour les cases du plan.

5 Validation

En resume

Sous MSVC, ThreadSanitizer n'existe pas : la correction concurrente ne peut pas être déléguée à un outil d'analyse dynamique. La stratégie adoptée combine quatre instruments : des tests déterministes sur composants isolés, un faux évaluateur qui rend les entrelacements reproductibles, des points de crochet qui forcent les collisions, et des tests de mutation qui vérifient que les gardes détectent réellement les erreurs. Les invariants d'arbre servent de contrôle final après chaque recherche. Ce chapitre inventorie ce que chaque test verrouille, et dit honnêtement ce qu'il ne couvre pas.

5.1 Tester la concurrence sans outil d'analyse

La difficulté d'un test concurrent classique est son irrépétabilité : un entrelacement fautif peut ne se produire qu'une fois sur mille, et un test qui passe ne prouve rien. Le projet a donc pris le problème par l'autre bout : au lieu d'espérer un entrelacement, on le *force* avec des points de synchronisation explicites, puis on vérifie les invariants.

Trois briques rendent cela possible. Le `ControlledEvaluator` simule le réseau : politique uniforme, valeur configurable, tailles de lot enregistrées, échec programmable, et un crochet `before_evaluate` qui peut bloquer l'appel sur une variable de condition. Le `WaveTestHooks` du collecteur offre deux points d'arrêt, `before_claim` et `before_tt_publish`, qui permettent de faire arriver deux workers exactement au même nœud. Enfin, le support de test `require_test` lève une exception au lieu d'appeler `assert` : les tests restent actifs en compilation `Release`, où `NDEBUG` aurait supprimé les assertions standard.

5.2 Inventaire des tests

Cible	Ce que le test verrouille
<code>search_timing</code>	accumulateurs par phase, mode désactivé sans horloge, fusion des workers
<code>evaluator_contract</code> <code>board_copy</code>	le faux réseau pilote réellement le MCTS ; dimensions d'entrée vérifiées FEN, Zobrist, tenseur, clé h7 et historiques identiques après copie, coup joué et annulé (roque, prise en passant, promotion, répétition)
<code>evaluation_cache</code>	snapshots stables, collisions forcées sur une même entrée, ordre des rejets, clear et probe après clear, troncature à 128 coups
<code>reservation</code>	propriété unique sous huit gardes concurrentes, déplacement, exception, compteurs nuls après libération
<code>node_state</code>	observabilité des états au repos, publication incohérente, valeurs non finies, visites de racine
<code>search_executor</code>	identités de threads stables, sous-ensemble actif, exception relayée après retour des workers, nouvel appel possible, arrêt idempotent
<code>wave_collection</code>	collision forcée entre deux collecteurs, publication depuis la table, terminaux avant table, lot partiel, budget borné, tenseur comparé au jeu réel de l'historique
<code>wave_search</code>	workers 2/4/8, lots 1/8, budgets exacts, racine terminale, erreurs de dimensions, immobilité pendant l'inférence, sérialisation de <code>update_root</code>
<code>multicore_stress</code>	100 recherches par configuration sur trois positions, racines neuves et réutilisées, alternance des workers, finales et cas terminaux
<code>selfplay_shared_core</code>	huit racines privées sur une instance MCTS, clés qui s'écrasent, deux parties complètes du manager
<code>selfplay_phase_bench</code>	référence de performance du chemin self-play partagé, comparée avant et après le chantier

5.3 Les invariants

Après toute recherche réussie, le parcours `inspect_tree` exige : tous les `n_in_flight` nuls ; aucun Pending au repos ; chaque réservation terminée ou annulée exactement une fois ; exactement un backup par simulation demandée ; aucun nœud développé deux fois ; des enfants uniques avec parents corrects et priors sommant à 1 ; le plateau d'entrée intact (FEN, hash, historiques, tenseur) ; et le coup principal légal sur le plateau racine. Ces contrôles sont exécutés après chaque appel dans le stress de la tâche 8, et à la main dans les tests ciblés.

Le budget exact mérite une phrase de plus. Il ne se vérifie pas sur les compteurs du réseau, parce qu'une simulation peut se terminer sur un nœud terminal ou sur un succès de table sans appel réseau. Il se vérifie sur `completed_simulations` d'une part, et sur la somme des visites de racine d'autre part, avec la comparaison $\sum \text{visites enfants} \leq \text{visites racine} \leq \sum + 1$ qui tolère les racines développées sans visite propre.

5.4 Les entrelacements forcés

Cinq scénarios défectueux sont reproduits à volonté.

1. **Collision.** Deux collecteurs sont synchronisés pour arriver au même `before_claim`. Une seule feuille réseau doit sortir, l'autre doit être une collision, sans enfant dupliqué, et les compteurs doivent revenir au repos quand les résultats sont détruits.
2. **Publication depuis la table.** Un worker bloque avant de publier ses enfants ; l'autre doit observer Pending et ne jamais parcourir une liste partielle. Après publication, une descente doit rejoindre un descendant, sans backup ni incrément de visites.
3. **Inférence bloquée.** Le faux réseau se bloque sur une variable de condition ; le compteur de descentes doit rester constant pendant le blocage, ce qui vérifie que les workers s'arrêtent bien à la barrière en phase de vagues.
4. **Exception au milieu d'un lot.** Un échec programmé de l'évaluateur doit laisser zéro Pending, zéro réservation, un arbre inspectable et une recherche ultérieure possible.
5. **Terminal avant table.** Une position de nulle par répétition ou par règle des cinquante coups doit être terminale même si une entrée de cache existe pour le même hash dans un autre contexte.

5.5 Les tests de mutation

Un test qui passe ne prouve pas qu'il détecte une erreur. Trois mutations volontaires ont donc été introduites, vérifiées comme détectées, puis retirées : la suppression d'une décrémentation du virtual loss (le test de fuite échoue), la neutralisation de la remise à zéro de Pending (le test d'exception échoue), et la neutralisation du CAS de propriété (le test de collision échoue). Cette pratique a aussi été appliquée au test de durée de vie Python : sans le `keep_alive` du binding, le sous-processus meurt par violation d'accès, ce qui prouve que le test discrimine.

5.6 Ce que la validation ne couvre pas

Trois limites sont assumées. D'abord, l'absence de ThreadSanitizer : la conception et les tests d'entrelacement remplacent un outil qui aurait pu trouver des courses non anticipées. Ensuite, les hooks forcent des entrelacements choisis, pas tous les entrelacements possibles ; un bug qui exige trois workers à un instant précis pourrait passer. Enfin, la qualité de recherche ne se teste pas par invariants : elle est mesurée par le banc de puzzles, un instrument statistique dont les limites sont décrites au chapitre 6.

6 Campagnes de mesure

En resume

Le multicœur apporte +8 à +26 % de simulations par seconde selon la position, avec un plafond dès deux workers et un p95 de latence qui ne se dégrade pas. La comparaison interleaved avec l'ancien binaire ne montre aucune régression mono-worker. Le self-play partage reste identique au bit près sur ses compteurs et ses visites. La qualité est non inférieure sur 2 500 puzzles (delta -0,28 point, IC95 [-0,8 ; +0,24]), et le test à temps égal se conclut à une égalité (364 contre 366). La mesure du régime réel du bot montre que la tranche de 64 simulations utilisée par la boucle UCI est le régime favorable, alors que la tranche de 8 du banc pénalise artificiellement huit workers. Le réglage `MCTS_WORKER_COUNT = 8` est activé.

6.1 Protocole et matériel

Toutes les campagnes utilisent le même modèle (2026_04_23_23h25_iter316_unsupervised.onnx, sha256 20a19f82e170), la même politique h0, une table de 8 192 entrées, un lot de 8 et $c_{\text{puct}} = 1,4$. La machine est un portable RTX PRO 2000 Blackwell (8 Gio, driver 595.95) avec un CPU de 16 cœurs logiques (Intel64 Family 6 Model 197). ONNX Runtime 1.24.3 utilise le provider CUDA. Les trois positions de référence sont l'ouverture, le milieu et la finale, définies en annexe C.

Le protocole de débit

Chaque configuration est mesurée 30 fois (campagne principale, 5×6 passages) ou 50 fois (campagne de queue), en alternant l'ordre des workers à chaque passage pour que la dérive thermique se répartisse équitablement. Le mode `-warmup-pool` conserve l'objet MCTS entre les mesures et remet arbre et table à froid hors chronométrage, de sorte que les médianes décrivent le régime établi et que le premier appel de chaque configuration (création des threads, première passe) soit mesuré séparément. Les sessions varient de 15 à 30 % entre elles : seules les comparaisons intra-session sont utilisées pour conclure.

6.2 Aucune régression mono-worker

Comparer le nouveau moteur à l'ancien pose un problème pratique : le binaire d'avant n'existe plus. Le pyd de l'époque (commit da29e7b) était committé dans Git ; il a été extrait dans un répertoire temporaire, le banc actuel y a été copié avec un `repl TypeError` pour l'appel sans `worker_count`, et les deux moteurs ont été mesurés en alternance, trois rounds de trois passages. L'alternance est essentielle : l'ancien moteur fluctue de 700 à 1 200 simulations par seconde selon l'état d'horloge du GPU, contre 700 à 820 pour le nouveau. Les rounds 1 et 2 donnent des rapports nouveau/ancien de 0,97 à 1,14 ; le round 3 est un pic de boost de l'ancien (1 018 sims/s en ouverture) que le nouveau ne suit pas, sans que cela traduise une régression. Les compteurs et les distributions de visites sont identiques. Conclusion : pas de régression d'infrastructure visible en mono-worker, ce qui était la condition pour interpréter le gain multicœur.

6.3 Débit et latence par nombre de workers

La campagne principale donne les médianes de la figure et du tableau suivants (chemin `step_analysis`, pool chaud, 30 observations).

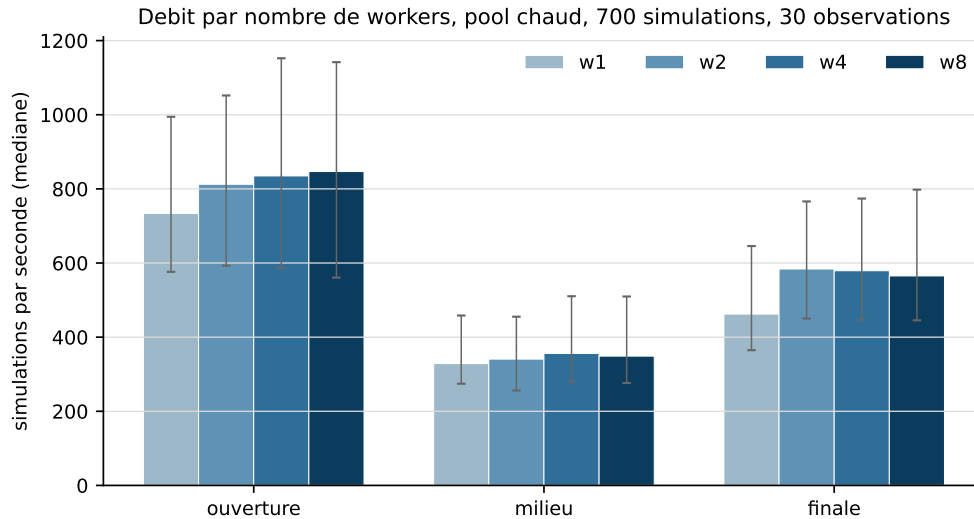


FIGURE 5 – Médianes de débit par position et par nombre de workers, pool chaud, 700 simulations, 30 observations par point (after .json). Les barres d'erreur vont du minimum au maximum observé.

Position	w1	w2	w4	w8	gain w8
ouverture	733,5	812,3	835,0	846,8	+15,4 %
milieu	328,7	340,7	356,1	348,8	+6,1 %
finale	462,2	583,8	579,2	565,1	+22,3 %

TABEAU 3 – Médianes de simulations par seconde, step_analysis, pool chaud. Le gain plafonne dès deux workers; mcts_search donne les mêmes ordres.

La campagne de queue, avec 50 observations par configuration et deux configurations seulement (1, 2 et 8 workers), confirme ces ordres de grandeur et permet de mesurer proprement la distribution.

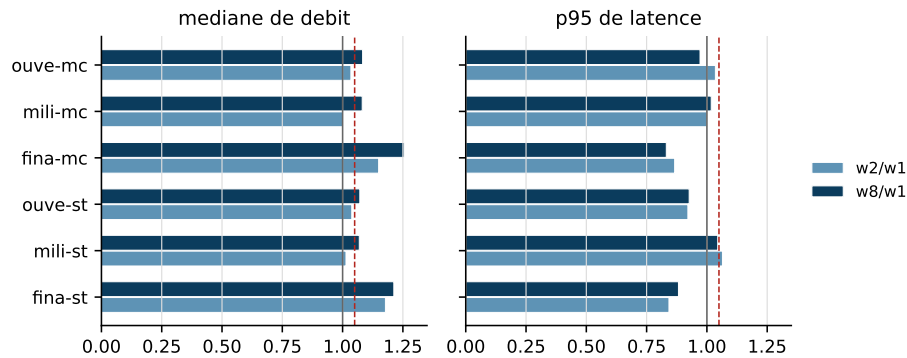
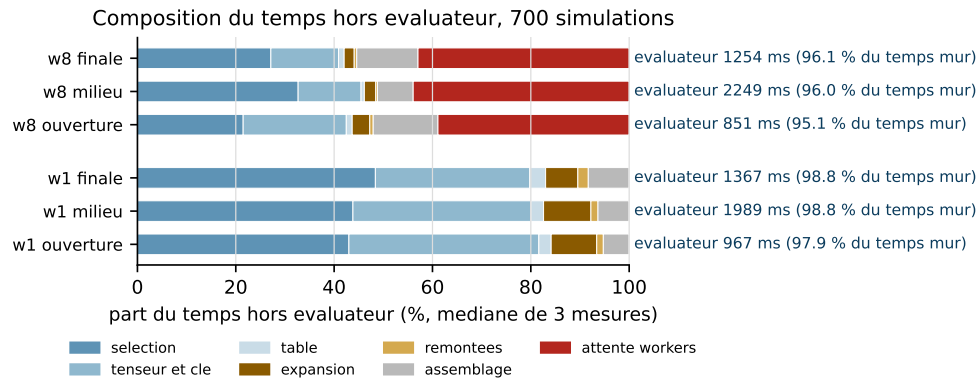
Le p95 de latence n'est pas le p95 de débit

Le critère d'acceptation porte sur la *latence* d'une recherche de 700 simulations. Or la latence est l'inverse du débit : le p95 de latence correspond au p5 du débit, c'est-à-dire aux recherches les plus lentes. Comparer le p95 du débit d'une configuration à celui d'une autre mesure la mauvaise queue, et donne l'impression inverse : on croit voir une dégradation alors que les recherches lentes s'amélioreraient. Le rapport de latence vaut $L_{95}(w_8)/L_{95}(w_1) = p_5(T_{w_1})/p_5(T_{w_8})$.

Avec la bonne définition, w8 franchit le critère partout : rapports de p95 de latence de 0,832 à 1,045 selon les six couples position-chemin, tous sous +5 %, et même améliorés sur la finale (−12 à −17 %). w2 franchit le critère partout sauf un point marginal (milieu step_analysis, +6,4 %), et capture déjà l'essentiel du gain de médiane. Le candidat retenu reste w8, qui a les meilleures médianes globales et la queue la plus stable.

6.4 Répartition par phase et remplissage

La campagne à chronométrages actifs (3 mesures par configuration) montre que l'inférence occupe 96,6 à 98,7 % du temps mur, dans toutes les configurations. En huit workers, l'attente de barrière ne pèse que 1,5 à 2,3 % : 24,3 ms en finale, 40,7 ms en milieu, 19,3 ms en ouverture pour des recherches de 1,3 à 2,3 secondes.

Rapports $w2/w1$ et $w8/w1$, campagne de queue, 50 observations par configuration**FIGURE 6** – Rapports $w2/w1$ et $w8/w1$ sur la campagne de queue (50 observations par configuration) : médiane de débit et p95 de latence. En dessous de la ligne des 5 %, le critère est franchi.**FIGURE 7** – Part du temps mur consacrée à l’inférence et au reste (somme des phases CPU et de l’attente), par position, en w1 et w8. Le reste est déjà minuscule en w1 ; le multicœur le réduit encore, mais ne peut pas toucher à l’inférence.

Le remplissage progresse légèrement : de 6,10 à 6,68 en finale et de 6,94 à 7,23 en ouverture. À nombre de positions évaluées constant ($nn_calls = 701$), cette hausse réduit le nombre d’appels GPU physiques, de 115 à 105 en finale et de 101 à 97 en ouverture. C’est le mécanisme même du gain, et il est spécifique aux vagues : en mono-worker, la collecte s’arrête à la première collision et ferme le lot partiel, alors qu’un worker de vague rend son permis et réessaie.

Le gain n’est pas un recouvrement CPU-GPU

On pourrait croire que les vagues font travailler le CPU pendant que le GPU calcule. C’est faux : la barrière est explicite, et le temps hors évaluateur *augmente* même légèrement en multicœur (19,5 ms en w1 contre 32,6 ms en w8 pour la finale), parce que l’attente de barrière s’ajoute à la collecte. Le gain vient uniquement de la réduction du nombre d’appels GPU, pas d’un chevauchement des phases.

La campagne principale et la campagne de phases ne chiffrent pas ce mécanisme de la même façon. Dans la campagne de phases, où le coût par appel est mesuré constant (11,88 ms en w1 contre 11,94 ms en w8), les dix appels économisés en finale expliquent la totalité du gain

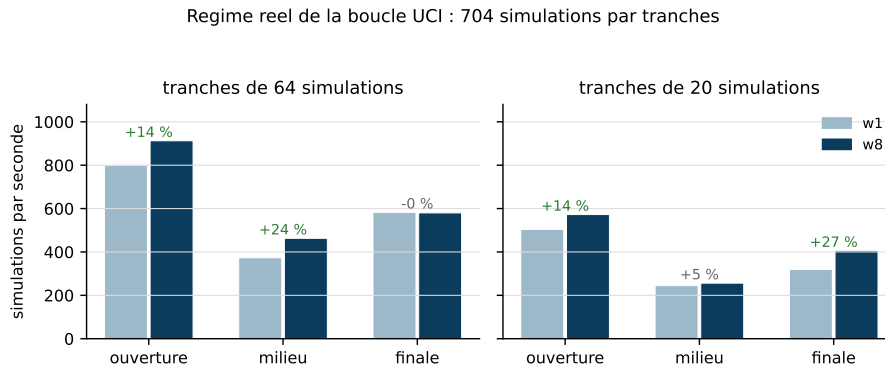


FIGURE 8 – Gain de w8 par rapport à w1 selon la taille de tranche. Aux deux tranches du bot (64 et 20 simulations), le gain est net sauf en finale à 64 ; à 8, non représenté ici, les arbres naissants font exploser les collisions et w8 recule.

de 8 %. Dans la campagne principale, le même couple affiche +22 %, que la seule réduction d'appels ne peut pas couvrir : la part résiduelle relève de la dérive entre sessions. Le tableau 7 du chapitre suivant sépare ces deux lectures ; il faut donc retenir le mécanisme comme certain et son ampleur comme approximative.

6.5 Le régime réel de la boucle UCI

Le bot n'appelle pas `step_analysis` une fois pour 700 simulations : sa boucle avance par tranches de `BATCH_SIZE = 64` simulations, sur un arbre et un pool persistants. Un diagnostic dédié a mesuré ce régime, avec des tranches de 64 et de 20, sur 704 simulations et trois répétitions.

Tranche	Position	w1 (sims/s)	w8 (sims/s)	gain
64	ouverture	799,1	912,5	+14 %
64	milieu	373,1	462,8	+24 %
64	finale	582,5	580,0	0 %
20	ouverture	503,9	572,4	+14 %
20	milieu	244,5	256,1	+5 %
20	finale	319,1	405,5	+27 %

TABLEAU 4 – Régime réel du bot : `step_analysis` répété par tranches, arbre et pool persistants, 704 simulations, 3 répétitions.

À la tranche de 8 simulations (celle du protocole à temps égal du banc puzzle), le rapport s'inverse : w8 tombe à 693 sims/s contre 857 pour w1, avec 970 collisions pour 800 simulations. Un arbre de huit simulations est trop jeune pour offrir huit branches distinctes ; les workers se disputent les quelques feuilles existantes, et le coût des collisions dépasse le gain de parallélisme. Ce phénomène n'existe plus à 64 simulations : la phase la plus jeune de chaque recherche est courte relativement au budget, et les collisions, bien que nombreuses (1 300 à 3 300 par 704 simulations), sont amorties. C'est le régime du bot, et c'est celui qui compte.

6.6 Non-régression du self-play

Le banc `selfplay_phase_bench` a été rejoué avec les mêmes réglages que la référence de la tâche 2 : huit plateaux, 100 vagues, 30 répétitions.

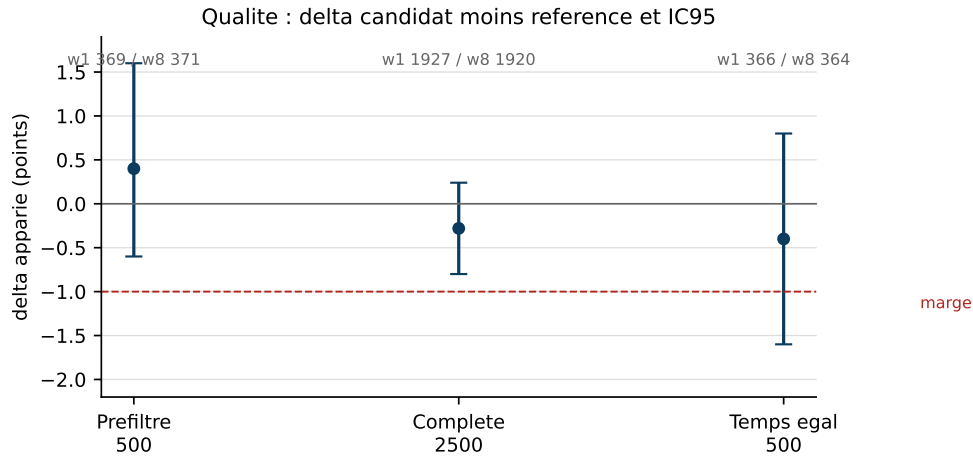


FIGURE 9 – Taux de résolution et intervalles de confiance bootstrap pour les trois campagnes. La ligne pointillée marque la marge de non-infériorité de -1 point sur l'intervalle du delta apparié.

Variante	Mesure	Avant	Après	Rapport
fake	collecte	0,738 ms	1,321 ms	+79 %
fake	traitement	4,390 ms	4,508 ms	+2,7 %
GPU	collecte	1,928 ms	2,003 ms	+3,9 %
GPU	traitement	340,97 ms	305,04 ms	-10,5 %

TABLEAU 5 – Banc self-play partagé, médianes de 30 répétitions. Les compteurs (800 appels, 100 lots, 103 succès de table en GPU) et les visites de référence sont identiques avant et après.

Le surcoût CPU des nouveaux états atomiques, du cache et des réservations vaut environ 0,58 ms pour 100 vagues en mode fake, soit 5,8 microsecondes par vague. Il est visible en relatif parce que le faux réseau ne coûte rien, et négligeable en absolu face aux 305 ms de réseau en GPU. Le traitement GPU mesuré est même plus rapide après qu'avant, ce qui relève de l'état d'horloge de la session, pas du chantier : l'important est que les compteurs et les visites soient identiques au bit près, ce qui prouve que la sémantique du self-play est intacte.

6.7 Qualité de recherche

La qualité est mesurée sur le banc de puzzles, au premier coup, avec le critère figé `reussi_recherche`. Trois campagnes ont été menées, toutes sur GPU, un seul processus, avec comparaison appariée et bootstrap à 20 000 tirages (seed 42).

Campagne	n	w1	w8	delta apparié (IC95)
Préfiltre (700 sims)	500	369	371	+0,4 pt [-0,6; +1,6]
Complete (700 sims)	2 500	1 927	1 920	-0,28 pt [-0,8; +0,24]
Temps égal (1,5 s)	500	366	364	-0,4 pt [-1,6; +0,8]

TABLEAU 6 – Qualité : réussites et delta candidat moins référence. Seule la campagne à simulations égales sert au verdict de non-infériorité ; le test à temps égal quantifie le gain pratique.

Trois lectures. D'abord, la campagne complète à 2 500 puzzles donne un delta de $-0,28$ point, avec vingt-quatre paires perdues contre dix-sept gagnées et un McNemar non significatif ($p = 0,35$). La borne basse $-0,8$ point est à l'intérieur de la marge de -1 point : le verdict formel est la non-infériorité. Par tranche de rating, les deltas vont de $-0,16$ à $-0,48$ point, sans concentration : c'est du bruit sur l'échantillon, pas un effet localisé. Ensuite, la répétition

du candidat prescrite par le plan en cas d'intervalle proche du seuil a été écartée par le propriétaire pour des raisons de durée : c'est la réserve principale de ce verdict. Enfin, la mesure à temps égal, avec un budget de 1,5 seconde par puzzle (la médiane observée des 700 simulations mono-worker), donne une égalité : 364 contre 366, intervalle $[-1,6; +0,8]$, indéterminé. Huit workers y terminent 472 simulations médianes contre 728, pour la raison mécanique décrite au chapitre 6.5 : le protocole à tranches de 8 pénalise les arbres naissants. Le taux de résolution, lui, reste égal. Le dépassement du dernier appel complet, en mode temps, a une médiane de 8,8 ms et un p95 de 51 ms, ce qui est très inférieur à la granularité d'une tranche.

6.8 La décision d'activation

Tous les critères du plan sont franchis : médiane améliorée de façon reproductible, p95 de latence sous +5 % partout, invariants vérifiés, non-infériorité qualité au seuil près, self-play identique. Le réglage `MCTS_WORKER_COUNT = 8` a donc été ajouté à côté de `MCTS_BATCH_SIZE = 8` dans `uci.py`, avec un test de transmission qui vérifie le cinquième argument réel de `step_analysis`, et un smoke UCI complet en sous-processus. Les réserves sont listées au chapitre 7.

7 Discussion

En resume

Le gain est modeste parce que le goulot n'était pas là où on le croyait : l'inférence occupait déjà 96 à 99 % du temps, donc paralléliser la préparation CPU ne pouvait pas donner plus que quelques pour cent de médiane, plus un effet de remplissage. Les critères d'acceptation sont franchis, mais la décision d'activer huit workers se paie d'une complexité réelle, et plusieurs menaces pèsent sur la généralisation des résultats. Ce chapitre les assume explicitement et en tire les leçons.

7.1 Pourquoi pas un facteur huit

La loi d'Amdahl [10] explique l'essentiel. Si une fraction p du temps est parallélisable, le gain maximal est $1/(1 - p)$. Ici, la mesure des phases donne $1 - p \approx 0,97$ à $0,99$: le gain maximal théorique d'un recouvrement serait donc de 1 à 3 %. Or le gain observé va de 7 à 26 % selon la position. L'écart ne vient pas d'un chevauchement entre CPU et GPU, puisque les phases alternent toujours et que l'attente de barrière augmente même légèrement le temps hors évaluateur. Il vient d'une réduction du nombre d'appels GPU, elle-même permise par un meilleur remplissage : en mono-worker, la collecte referme le lot à la première collision (`mcts_batch.cpp`, sortie sur Pending), alors qu'un worker de vague rend son permis et réessaie dans la limite des tentatives (`mcts_wave.cpp`). À nombre de positions évaluées constant, les vagues déclenchent donc moins d'appels physiques.

L'ampleur de cet effet est mesurable, mais elle ne suffit pas à expliquer tous les chiffres de la campagne principale. Sur la campagne à chronométrages actifs, où le coût par appel est mesuré constant, la réduction d'appels explique la totalité d'un gain de 8 % en finale. Sur la campagne principale, mesurée dans une autre session, le même couple affiche 22 % : la réduction d'appels n'en explique alors qu'une partie, et le reste relève de la dérive entre sessions, qu'il serait abusif d'attribuer au remplissage. Le tableau 7 sépare les deux lectures.

Position	remplissage w1 → w8	appels GPU w1 → w8	gain de débit w8/w1	part des appels dans le gain
ouverture	6,94 → 7,23	101 → 97	+15 %	≈ 30 %
milieu	4,87 → 5,19	144 → 135	+6 %	≈ 100 %
finale	6,10 → 6,68	115 → 105	+22 %	≈ 43 %

TABLEAU 7 – Décomposition du gain, `step_analysis`, pool chaud, 700 simulations, 30 observations (`after.json`). La dernière colonne estime la part du gain de débit explicable par les appels GPU économisés, au coût par appel de la campagne de phases (≈ 11,9 ms en finale). Elle dépasse 100 % pour le milieu, où le gain de débit est du même ordre que le bruit de session.

Cette décomposition est une estimation, pas une mesure directe : les appels sont comptés, mais leur coût par appel est emprunté à une autre session. Elle suffit néanmoins à écarter l'explication d'un recouvrement CPU-GPU et à montrer que le remplissage est le mécanisme, sans prétendre chiffrer sa part exacte.

Il y a une seconde raison, plus profonde, pour laquelle le plafond est bas : le GPU était déjà largement occupé pendant la collecte, puisque celle-ci ne dure que quelques dizaines de millisecondes face à une inférence de plusieurs centaines. L'alternance des phases coûte peu (1,5 à 2,3 % d'attente de barrière) parce que la collecte elle-même est courte. Autrement dit, l'architecture batchée était déjà proche de sa limite : le GPU est le goulot, et il le reste.

7.2 Ce qui a marché, ce qui a déçu

Ce qui a marché : la finale, où le gain atteint +22 % de médiane et où le p95 de latence s'améliore même de 12 à 17 %, parce que les recherches lentes y sont justement celles qui bénéficiaient le plus du remplissage complet. L'égalité de qualité tient sur 2 500 puzzles, et le self-play n'a pas bougé. La correction est démontrable par invariants et tests de mutation, pas seulement par empirisme.

Ce qui a déçu : le milieu de jeu, où le gain de médiane reste sous 8 %, et le plafond dès deux workers, qui dit clairement que la quatrième et la huitième unité de calcul ne servent presque à rien. La tranche de 8 simulations, utilisée par le protocole à temps égal, est même un régime où huit workers reculent de 19 % : sur des arbres de huit simulations, les collisions dominent. C'est une propriété du couple (taille de tranche, nombre de workers) qui mérite d'être connue avant d'activer le multicœur dans un autre contexte que le bot.

7.3 Menaces sur la validité

1. **Dérive d'horloge et thermique.** Les sessions varient de 15 à 30 %. Tout a été mesuré en alternance pour compenser, mais les valeurs absolues ne sont pas transférables telles quelles d'une machine à l'autre.
2. **Queue de latence sur 50 observations.** Le p95 y est estimé sur 50 valeurs par configuration ; c'est mieux que les 6 valeurs par passage de la campagne principale, mais cela reste un estimateur de queue imparfait. Le plan lui-même prévient qu'un p95 instable exige de nouvelles observations.
3. **Échantillon de puzzles.** Le banc est un sous-échantillon régulier de 2 500 puzzles sur 5 000, stratifié par rating. Il mesure le premier coup d'une position tactique, pas la force en partie complète. Le verdict de non-infériorité est un argument de qualité de recherche, pas une mesure Elo.
4. **Arbre froid.** Toutes les mesures partent d'un arbre neuf. En partie réelle, `update_root` conserve le sous-arbre et une partie des simulations évite le réseau. Par la loi d'Amdahl, le gain du multicœur s'en trouverait réduit. Le cas chaud n'est pas mesuré.

5. **Table du bot.** Le bot utilise 4 000 000 d'entrées (environ 4,16 Gio), les mesures 8 192. Un taux de succès de table plus élevé réduirait encore la fraction accélérée.
6. **Répétition écartée.** La campagne de 2 500 puzzles n'a pas été répétée; l'incertitude résiduelle de qualité va jusqu'à -0,8 point. C'est la limite la plus importante du verdict, et elle est consignée comme telle dans le rapport de résultats.
7. **Mesures GPU seulement.** Le multicœur a été mesuré sur le GPU. Sur CPU, le rapport entre collecte et inférence est très différent, et le gain serait probablement plus important; il n'a pas été caractérisé.

7.4 Le coût d'ingénierie

Dix tâches, cinq composants, quatorze cibles de test et plus de deux cents tests Python : le chantier est lourd pour un gain de l'ordre de 10 %. Faut-il le regretter ? Trois arguments disent non. D'abord, la partie non parallélisable était déjà optimisée ; sans ce chantier, aucun gain supplémentaire n'était disponible côté ordonnancement, et les mesures le démontrent. Ensuite, les composants produits (cache par snapshots, évaluateur injectable, garde RAIL, exécuter persistant, collecteur borné) sont exactement les fondations de la production continue : les abstractions sont réutilisées, pas jetées. Enfin, la discipline de test a produit une base de référence mesurée (chronométrages, harnais, comparaison appariée) qui sert déjà à d'autres chantiers, comme l'étude des coûts CPU-GPU.

Le risque, lui, est réel : une structure concurrente qu'on ne comprend plus est une dette. C'est pourquoi l'observabilité a été traitée comme une fonctionnalité de premier ordre : compteurs de collisions, d'états Pending, de fuites de virtual loss, messages de parcours. Un arbre corrompu se voit au lieu de se deviner.

7.5 Leçons

1. **Mesurer avant d'optimiser.** L'intuition initiale, « le CPU prépare pendant que le GPU calcule », était fautive dans ce régime : les deux alternaient, et le GPU dominait à 97 %. La chronométrie par phase, ajoutée en première tâche, a redéfini l'objectif avant d'écrire la moindre ligne de concurrence.
2. **Tester la concurrence par la conception.** Sans ThreadSanitizer, la seule stratégie tenable est de réduire la surface concurrente à quelques états atomiques et une garde, puis de la harceler avec des hooks et des mutations.
3. **Un faux réseau vaut mille GPU.** Le ControlledEvaluator a rendu testables les erreurs, les blocages et les interleavings que le GPU ne reproduit jamais deux fois.
4. **Comparer en alternance.** Toute comparaison de performance sur une machine à horloge variable exige un protocole apparié ; l'écart entre l'ancien et le nouveau binaire était invisible sans cela.
5. **Attention aux définitions des quantiles.** Le p95 de latence est le p5 de débit ; s'être trompé de queue aurait fait refuser un réglage qui passe les critères.
6. **Un test peut être faux.** Le premier test d'équivalence du pool chaud comparait la mauvaise référence ; le refaire correctement a transformé un échec en preuve.

8 Conclusion et perspectives

En resume

La recherche multicœur par vagues est livrée, testée, mesurée et active dans le bot. Elle apporte +8 à +26 % de débit selon la position, sans dégradation de la queue de latence ni de la qualité de recherche, et sans régression du self-play. Le plafond observé est intrinsèque : l'inférence réseau est le goulot. La suite naturelle n'est donc pas davantage de workers, mais l'assainissement de l'appel à l'évaluateur et, seulement après, la production continue.

8.1 Bilan

Le chantier a transformé une boucle strictement sérielle en une architecture à deux phases, avec une surface concurrente volontairement réduite : quatre états de nœud atomiques, un compteur de virtual loss, des snapshots de table sous verrous rayés, et une garde RAIL qui possède le chemin réservé. La démonstration de correction repose sur des invariants vérifiés après chaque recherche, des entrelacements forcés et des tests de mutation, faute de ThreadSanitizer sous MSVC.

Sur le plan des performances, le verdict est mesuré et le mécanisme est compris : deux workers capturent l'essentiel du gain, huit l'emportent légèrement sur les médianes et sur la queue, et la zone gagnante est la finale. Sur le plan de la qualité, la non-infériorité est acquise sur 2 500 puzzles, avec une réserve assumée : la répétition du candidat n'a pas été faite.

8.2 Ce que la production continue peut reprendre

L'étape 2 de la conception, la production continue, ferait descendre les workers pendant les inférences et les remontées, au lieu de les arrêter à une barrière. Tous ses composants existent déjà : contextes privés, résultats immuables, garde RAIL, propriété Pending, cache thread-safe, évaluateur contrôlable, exécuteur persistant. Ce qui manque est la synchronisation des statistiques et de la structure d'arbre pendant que d'autres threads lisent.

Les mesures de ce rapport permettent d'en estimer l'enjeu : l'attente de barrière ne représente que 1,5 à 2,3 % du temps, et le remplissage plafonne déjà vers 7. Autrement dit, la production continue ne peut pas créer beaucoup plus de parallélisme utile à lot de 8 ; son intérêt dépend de la possibilité de former des lots plus gros, ce qui est précisément la question du chantier suivant.

8.3 Assainir l'appel à l'évaluateur

L'étude 2026-09-19-cout-calcul-cpu-gpu.md a mis en évidence un écart inattendu : le banc self-play évalue huit positions en 3,0 ms, alors que la recherche paie 10 à 14 ms par appel à remplissage comparable. La différence est attribuée à la forme variable des lots et aux allocations par appel côté ONNX Runtime. Le test T1b de cette étude (forme de lot fixe, buffers réutilisés) est l'expérience la plus rentable identifiée à ce jour, avec un potentiel de facteur 3 à 5 sur l'appel. Si elle se confirme, la production continue devient beaucoup plus intéressante, parce que de plus gros lots deviendront utiles. C'est l'ordre logique : d'abord l'appel, ensuite l'ordonnancement.

8.4 Autres pistes consignées

Mesurer le cas chaud (arbre réutilisé, table de 4 000 000 d'entrées) pour chiffrer la part de simulations qui évitent le réseau en partie réelle. Revoir la taille de table maintenant que la recherche tourne à des budgets plus élevés. Envisager FP16 ou TensorRT pour le réseau, après

T1b. Pour le self-play, une file d'inférence centrale pourrait lisser les vagues GPU, mais les mesures montrent qu'il est déjà efficace par parties indépendantes.

8.5 Mot de la fin

Ce chantier aura surtout montré une chose : la performance d'un moteur d'échecs neuronal ne se joue pas là où l'intuition la place. Le multicœur était la suite logique du batching, et il rend bien ce qu'il peut rendre, environ 10 % de médiane, parce que 97 % du temps est déjà dans le GPU. La vraie réserve de performance est dans la plomberie de l'appel à l'évaluateur, pas dans le nombre de cœurs. Mesurer avant d'optimiser, et mesurer le régime réel plutôt que le régime commode, aura été la boussole de tout le chantier.

A Chronologie des commits

Les commits cités dans le rapport, dans l'ordre chronologique. La colonne *chantier* distingue la préhistoire (batching et table h0) du chantier multicœur proprement dit.

Préhistoire : batching et table h0

Commit	Date	Objet
0fb11d5	2026-09-14	Boucle batchée avec virtual loss
383d938	2026-09-14	GPU et batching activés dans le moteur UCI
0495e69	2026-09-14	Batching dans le jeu local et le tournoi
2adae81	2026-09-14	Alignement des compteurs de terminaux
5243f2d	2026-09-14	Mesure du remplissage des lots
e78f094	2026-09-14	Balayage du débit GPU par taille de lot
65db204	2026-09-14	Validation et activation du lot 8
7de9779	2026-09-15	Clé sémantique des évaluations (h0)
3c3d1c7	2026-09-15	Sélection de h0 et rapport TT

Chantier multicœur

Commit	Date	Objet
df3d3b9	2026-09-16	Révision de la conception multicœur
da29e7b	2026-09-16	Plan des dix tâches
4b4f92f	2026-09-16	T1 : chronométrages optionnels
fe63dbe	2026-09-16	T2 : évaluateur injectable et banc self-play
a0a182d	2026-09-16	T3 : cache par snapshots et bandes
a303d96	2026-09-16	T4 : états de nœud et réservations RAI
540e5ee	2026-09-16	T5 : workers persistants
2772087	2026-09-16	T6 : collecte parallèle
3b42dd2	2026-09-16	T7 : coordinateur et API
ef327e2	2026-09-17	T8 : stress et self-play partagé
9ad2dc8	2026-09-17	T8 : cases du plan
d4e2712	2026-09-19	Nettoyage des caches Python et spec
7bde9e3	2026-09-19	T9 : banc multicœur et comparaisons appariées
c6ac1c0	2026-09-19	T9 : cases du plan
7cef664	2026-09-19	Durcissement des verdicts et statuts de campagne
9840fa8	2026-09-19	Alternance de l'ordre des workers
f752e8b	2026-09-19	Étude des coûts CPU et GPU
412b5d4	2026-09-19	T10 : rapport, activation UCI
0aaeeb1	2026-09-19	T10 : cases du plan

B Fichiers et responsabilités

Fichier	Responsabilité
src/search_timing.hpp/.cpp	Phases, accumulateurs par thread, PhaseTimer sans coût désactivé
src/evaluator.hpp	Contrat d'évaluation par lot, sans dépendance ONNX
src/evaluation_cache.hpp/.cpp	Table par snapshots, verrous rayés, ordre des rejets h0
src/mcts_reservation.hpp/.cpp	NodeState, garde PathReservation, virtual loss
src/search_executor.hpp/.cpp	Pool de workers persistant, rendez-vous C++17
src/mcts_wave.hpp/.cpp	LeafWork, collecte parallèle, coordinateur des vagues
src/mcts.hpp/.cpp	Structures de nœud, sélection, expansion, backup, analyse continue
src/mcts_batch.cpp	Boucle séquentielle et boucle batchée historiques
src/mcts_observe.cpp	Compteurs, inspect_tree
src/bindings.cpp	Interface pybind11, worker_count, clear_evaluation_cache
src/selfplay_manager.cpp	Self-play OpenMP, batching par parties, dépend d'Evaluator
tests/cpp/controlled_evaluator.hpp	Faux réseau contrôlable
tests/cpp/test_support.hpp	require_test actif en Release
tests/cpp/test_*.cpp	Tests de composants, d'entrelacement et d'intégration
tests/cpp/selfplay_phase_bench.cpp	Banc de référence du chemin self-play partagé
python_src/search_bench.py	Banc de débit UCI, workers, pool chaud, chronométrages
python_src/puzzle_bench.py	Banc de qualité par puzzles, mode temps, jambe GPU, sidecar
python_src/bench_metrics.py	Noyau de scoring, agrégation, rapport
python_src/multicore_comparison.py	Comparaison appariée, bootstrap, verdicts
python_src/uci.py	Bot UCI, MCTS_BATCH_SIZE et MCTS_WORKER_COUNT
docs/superpowers/specs/2026-09-14-mcts-multicore-design.md	Spec de conception
docs/superpowers/plans/2026-09-16-mcts-multicore-waves.md	Plan des dix tâches
docs/superpowers/specs/2026-09-16-multicore-waves-results.md	Rapport de mesures et décision
docs/superpowers/specs/2026-09-19-cout-calcul-cpu-gpu.md	Étude des coûts CPU et GPU

C Reproduction des campagnes

Toutes les commandes s'exécutent depuis la racine du dépôt. La variable \$m désigne le modèle de référence :

```

1 $m = 'python_src/checkpoints_onnx/2026_04_23_23h25_iter316_unsupervised.onnx'
2 uv run pytest python_src/tests -q
3 ctest --test-dir build -C Release --output-on-failure

```

Listing 4 – Variables et construction.

Débit et phases

```

1 uv run python python_src/search_bench.py --model $m --gpu \
2   --simulations 700 --passages 5 --repetitions 6 --batch-sizes 8 \
3   --worker-counts 1 2 4 8 --cache-history-depths 0 --tt-size 8192 \
4   --warmup-pool --out-json out/multicore/after.json \
5   --out-rapport out/multicore/after.md
6
7 uv run python python_src/search_bench.py --model $m --gpu \
8   --simulations 700 --passages 1 --repetitions 3 --batch-sizes 8 \
9   --worker-counts 1 2 4 8 --cache-history-depths 0 --timings \
10  --warmup-pool --out-json out/multicore/phases.json \
11  --out-rapport out/multicore/phases.md

```

Listing 5 – Étape 10 du plan, campagne principale et campagne de phases.

La campagne de queue utilise les mêmes options avec `-passages 50` `-repetitions 1` `-worker-counts 1 2 8` et `-out-json out/multicore/queue.json`.

Qualité

```

1 # prefiltre 500 puzzles
2 uv run python python_src/puzzle_bench.py --model $m \
3   --banc data/puzzles_bench.txt --gpu --travailleurs 1 \
4   --search-workers 1 --batch-size 8 --cache-history-depth 0 \
5   --simulations 700 --limite 500 \
6   --out-csv out/multicore/prefilter-w1.csv
7
8 # campagne complete 2500 (idem pour --search-workers 8)
9 uv run python python_src/puzzle_bench.py --model $m \
10  --banc data/puzzles_bench.txt --gpu --travailleurs 1 \
11  --search-workers 8 --batch-size 8 --cache-history-depth 0 \
12  --simulations 700 --limite 2500 \
13  --out-csv out/multicore/full-w8.csv
14
15 # temps egal, budget median observe de 700 simulations mono-worker
16 uv run python python_src/puzzle_bench.py --model $m \
17  --banc data/puzzles_bench.txt --gpu --travailleurs 1 \
18  --search-workers 8 --batch-size 8 --cache-history-depth 0 \
19  --search-seconds 1.5 --limite 500 \
20  --out-csv out/multicore/temps-w8.csv
21
22 # comparaison appariee (sidecar obligatoire a cote de chaque CSV)
23 uv run python python_src/multicore_comparison.py \
24  --baseline out/multicore/full-w1.csv \
25  --candidate out/multicore/full-w8.csv \
26  --out-report out/multicore/full-comparison.md --seed 42

```

Listing 6 – Préfiltre, campagne complète et temps égal.

Self-play partagé

```

1 build/Release/selfplay_phase_bench.exe --fake \
2   > out/multicore/selfplay-after-fake.json
3
4 $env:PATH = ".venv\Lib\site-packages\torch\lib;$env:PATH"
5 build/Release/selfplay_phase_bench.exe --model $m --gpu \
6   > out/multicore/selfplay-after-gpu.json

```

Listing 7 – Banc self-play. Le dossier `torch/lib` doit être dans `PATH` pour que le provider CUDA trouve `cublasLt`.

Positions de référence

Nom	Rôle	FEN
ouverture	attaque italienne	r1bqkbnr/1ppp1ppp/p1n5/1B2p3/4P3/ 5N2/PPPP1PPP/RNBQK2R w KQkq - 0 4
milieu	position de Kiwipete	r3k2r/p1ppqpb1/bn2pnp1/3PN3/1p2P3/ 2N2Q1p/PPPBPPPP/R3K2R w KQkq - 0 1
finale	finale de tours	8/2p5/3p4/KP5r/1R3p1k/8/4P1P1/8 w - - 0 1

D Glossaire

MCTS

Recherche arborescente de Monte-Carlo. Ici, un PUCT : chaque itération descend l'arbre en maximisant un score qui mélange valeur moyenne et prior, atteint une feuille, l'évalue par le réseau et remonte la valeur sur le chemin.

PUCT

Formule de sélection utilisée : exploitation W/N plus exploration $c_{\text{puct}} \cdot P \cdot \sqrt{N_p} / (1 + N + V)$.

FPU

First Play Urgency. Un enfant jamais visité hérite de la valeur du parent, réduite d'une pénalité, au lieu de zéro.

Virtual loss

Compteur V posé sur le chemin d'une descente en cours. Il ne modifie que le terme d'exploration, jamais Q .

Table de transposition (TT)

Cache des évaluations par position.

h0

Politique de clé de la table qui vérifie le compteur des cinquante coups, le contexte courant et l'historique courant.

Snapshot

Copie locale d'une entrée de table, prise sous verrou, qui reste valide même si l'entrée est écrasée ensuite.

Bande (stripe)

Mutex parmi un groupe qui protège un sous-ensemble d'entrées. Ici, 4096 bandes au plus, déduites de l'index physique.

RAII

Resource Acquisition Is Initialization. La garde PathReservation libère ses compteurs et son état dans son destructeur, y compris sur exception.

État de nœud

Unexpanded, Pending, Expanded ou Terminal. La propriété d'une feuille est une transition atomique entre les deux premiers.

Remplissage

Nombre de positions réellement évaluées par appel réseau. Inférieur à la taille de lot quand un lot partiel se ferme.

Lot partiel

Lot qui part avec moins de positions que la capacité, parce que des collisions ont fermé la collecte. Il garantit le progrès.

Quiescence

État d'un arbre au repos : aucun Pending, aucun `n_in_flight` non nul, aucune réservation vivante.

Pool chaud

Mode du banc où l'objet MCTS est conservé entre les mesures, les threads étant créés une seule fois ; arbre et table sont remis à froid hors chronométrage.

Sidecar

Fichier JSON de contexte associé à un CSV de campagne : hashes, budget, provider, workers. La comparaison refuse de comparer sans lui.

Bootstrap apparié

Méthode qui rééchantillonne les paires de puzzles avec remise pour estimer l'intervalle de confiance de la différence de taux. Ici, 20 000 tirages, seed 42.

McNemar

Test de comparaison appariée sur les paires discordantes.

p95 de latence

95^e centile du temps d'une recherche. Comme la latence est l'inverse du débit, il correspond au 5^e centile du débit.

Self-play

Le moteur joue contre lui-même pour produire les données d'entraînement. Il utilise plusieurs parties indépendantes en OpenMP et un lot GPU de taille 512.

Production continue

Étape 2 de la conception : les workers descendent pendant les inférences et les remontées, au lieu de s'arrêter à une barrière.

Références

- [1] D. Silver et al. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. arXiv :1712.01815, 2017.
- [2] LapZero-Chess. *Batching de la recherche MCTS avec virtual loss : conception*. docs/superpowers/specs/2026-09-11-mcts-batching-design.md, 2026.
- [3] LapZero-Chess. *Recherche MCTS multicoeur : conception*. docs/superpowers/specs/2026-09-14-mcts-multicore-design.md, révision du 16 septembre 2026.
- [4] LapZero-Chess. *MCTS multicoeur par vagues : plan d'implémentation*. docs/superpowers/plans/2026-09-16-mcts-multicore-waves.md, 2026.
- [5] Leela Chess Zero. *Sources du moteur, branche master*, commit 043e7233e115b1ff44f892513b91b6fc1fa49eb6. <https://github.com/LeelaChessZero/lc0>.
- [6] KataGo. *Sources du moteur*, commit 76369bf535b5fe9310ef93fd3f108b93a44e7f. <https://github.com/lightvector/KataGo>.
- [7] G. Chaslot, M. Winands, H. van den Herik. *Parallel Monte-Carlo Tree Search*. CG 2008.
- [8] Stockfish. *Sources du moteur*. <https://github.com/official-stockfish/Stockfish>.
- [9] L. Lamport. *How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs*. IEEE Transactions on Computers, 1979.
- [10] G. Amdahl. *Validity of the single processor approach to achieving large scale computing capabilities*. AFIPS, 1967.